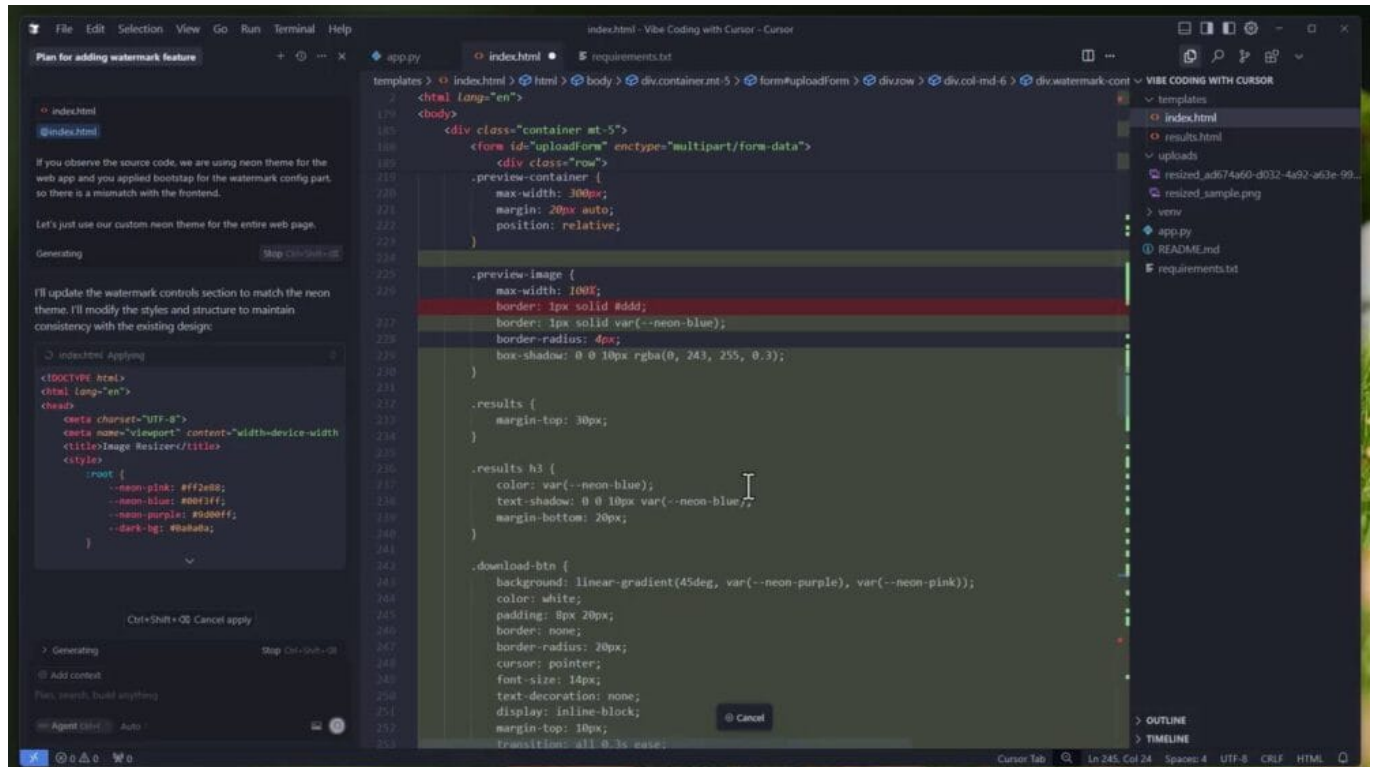


Welcome to this beginner-friendly course on vibe coding with the Cursor AI Code Editor. Over the next several lessons, you will watch and then replicate a complete workflow where you describe what you want and the AI handles most of the heavy lifting. By the end of the course, you will have built a working Flask web application that uploads images, resizes them, and adds watermarks.



I'm Muminjon, and we'll work through the full build together. The first part of the course unpacks the why, what, and how of the Cursor AI Code Editor. From there, the course moves into a live build loop so you can follow along and later apply the same process to other applications or keep iterating on this project.

Prerequisites

Before you start, make sure you are comfortable with a few basics. None of the optional items are required, but they will help you move faster through the course.

- **Python 3 basics:** creating and using functions, creating instances of classes, and working with a standard Python environment.
- **Git (optional):** a local Git repository is strongly recommended. AI tools can occasionally break your project, and a Git history lets you roll back changes and track what the assistant did on each iteration.
- **Flask (optional):** familiarity with the Flask web framework makes the code base easier to follow.
- **Pillow imaging library (optional):** knowing the basics of Pillow helps when reading the image processing code.
- **Curiosity and willingness to iterate:** vibe coding is an iterative loop with the AI, so a willingness to experiment matters more than deep expertise.

Course objectives

By the end of the course you will be able to:



1. Understand the mindset behind vibe coding.
2. Use Cursor's five core features, from code base indexing through to full agent mode.
3. Craft prompts that produce the output you actually want.
4. Put the theory into action by building, testing, and securing a Flask-based image resizer.
5. Walk away with a workflow you can apply to any future project.

Course topics

The course is organized into the following topics:

- Vibe coding fundamentals.
- Installing and configuring Cursor with the right privacy guardrails.
- Prompt structure, including the magic of the `.cursorrules` file.
- A live build loop covering planning, generating, testing, and refining the app in rapid cycles.
- Plugging security holes and generating documentation for the app in seconds.
- Deployment tips, followed by a small challenge where you extend the project with your own vibe-coded feature.

About Zenva Academy

A quick word about Zenva before you begin. Zenva is an online learning academy with over a million students, offering a wide range of courses for people who are just starting out as well as those who want to learn something new. For each course you can choose how you learn: watch the video tutorials, read the accompanying lesson summaries, or follow along using the included project files.

Getting started

With the goals, prerequisites, and topics in mind, you are ready to begin. The next lesson dives into vibe coding fundamentals, setting up the mindset and concepts that the rest of the course builds on.



In this lesson, we introduce the fundamentals of **vibe coding**, a modern way of building software in which a large language model works alongside you inside your code editor. You will learn what vibe coding is, when it makes sense to use it, the core features of the Cursor AI code editor, how to write effective prompts, how to guide the AI with a `.cursorrules` file, and the most common pitfalls to watch out for.

What is vibe coding?

Vibe coding is best understood as pair programming where your partner is a large language model integrated directly into your IDE. You point out the destination and the AI drives most of the route, handling boilerplate, scaffolding, and routine edits so you can stay focused on higher-level decisions.

In a pilot project run by The Pragmatic Engineer community, teams reported shipping features roughly three times faster with this approach. They also noticed that the draining context switches between code, documentation, and external resources like Stack Overflow almost disappeared, because most of that lookup work happened inside the editor itself.

Vibe coding is especially useful in a few specific situations:

- Putting together a quick Minimum Viable Product (MVP).
- Making a first pass at a greenfield module where the design is still fluid.
- Running a sweeping refactor of older code.

It is important to treat vibe coding as a copilot, not an autopilot. You still own the overall architecture, the security of your application, and the testing strategy. The AI can accelerate these tasks, but the final responsibility for them remains with you.

Core features of the Cursor AI code editor

Cursor is built around five core features that together enable the vibe coding workflow:

1. **Codebase index:** Cursor ingests your repository and indexes it for semantic search and retrieval-augmented generation. Because the editor has your actual files in context, it can answer questions with real snippets from your codebase instead of generic suggestions.
2. **Chat or inline ask:** You can highlight a block of code, trigger the command, and chat directly about the lines you do not understand or want to change.
3. **Composer:** If you need to rewrite a chunk of code, rename variables, or rename class names across a file, you can do it from a single prompt rather than editing each occurrence by hand.
4. **Agent mode:** You issue a high-level goal and Cursor breaks it down into steps, writes the code, and can even patch failing tests as it goes.
5. **Multiline completion:** As you type, you can press tab to accept entire functions that materialize as complete snippets, not just single-line autocompletions.

Together, these pillars reduce boilerplate, free up mental bandwidth, and help you stay in flow while coding.

Creating effective prompts

Good prompts are half the battle when vibe coding. A reliable structure for a strong prompt has four parts:

1. **Goal:** The concrete action you want performed, for example “create an upload endpoint in my web application.”
2. **Context:** Why the task matters and where it fits. This is where you mention the purpose of the app, the libraries in use, and any relevant stylistic conventions.



3. **Constraints:** The technical requirements the answer must respect, such as “use Python 3.12,” “use Flask for the endpoint,” or any internal style guide the team follows.
4. **Format:** How you want the answer returned, for instance “return only the code block” so that you can paste it straight into your main codebase.

You can start with a broad prompt, run it, and then progressively refine it as the result gets closer to what you want. For the upload endpoint example, you might add constraints like limiting file size to 2 MB or 10 MB, or streaming a preview in base64 format. You can also anchor the AI by referencing specific files directly, such as `app.py`, `templates/index.html`, or `preview.js`, so that Cursor pulls exact lines from those files rather than hallucinating them.

The `.cursorrules` file

Cursor also supports a small but powerful configuration file called `.cursorrules`. You place it at the root of your repository and Cursor treats it as authoritative guidance for every interaction within that project. In effect, it is a combination of style guide, dependency whitelist, and privacy valve, all rolled into one file.

Some examples of rules you might include are:

- “Prefer async Flask blueprint pattern” so that every new route is generated as asynchronous.
- “Use the Pillow imaging library and avoid cv2 (OpenCV)” to steer the AI away from dependencies you do not want.
- “Write tests with PyTest” and “all responses must include PyTest tests” when you want new code to ship with unit tests.
- “Ignore the `node_modules` folder and `.env` file” to blacklist secret or noisy files so that their contents never leave your machine.

Pitfalls and safeguards

Cursor is a powerful tool, but power tools can cut you if you are not careful. Before relying on the AI in real projects, it is worth understanding the most common failure modes:

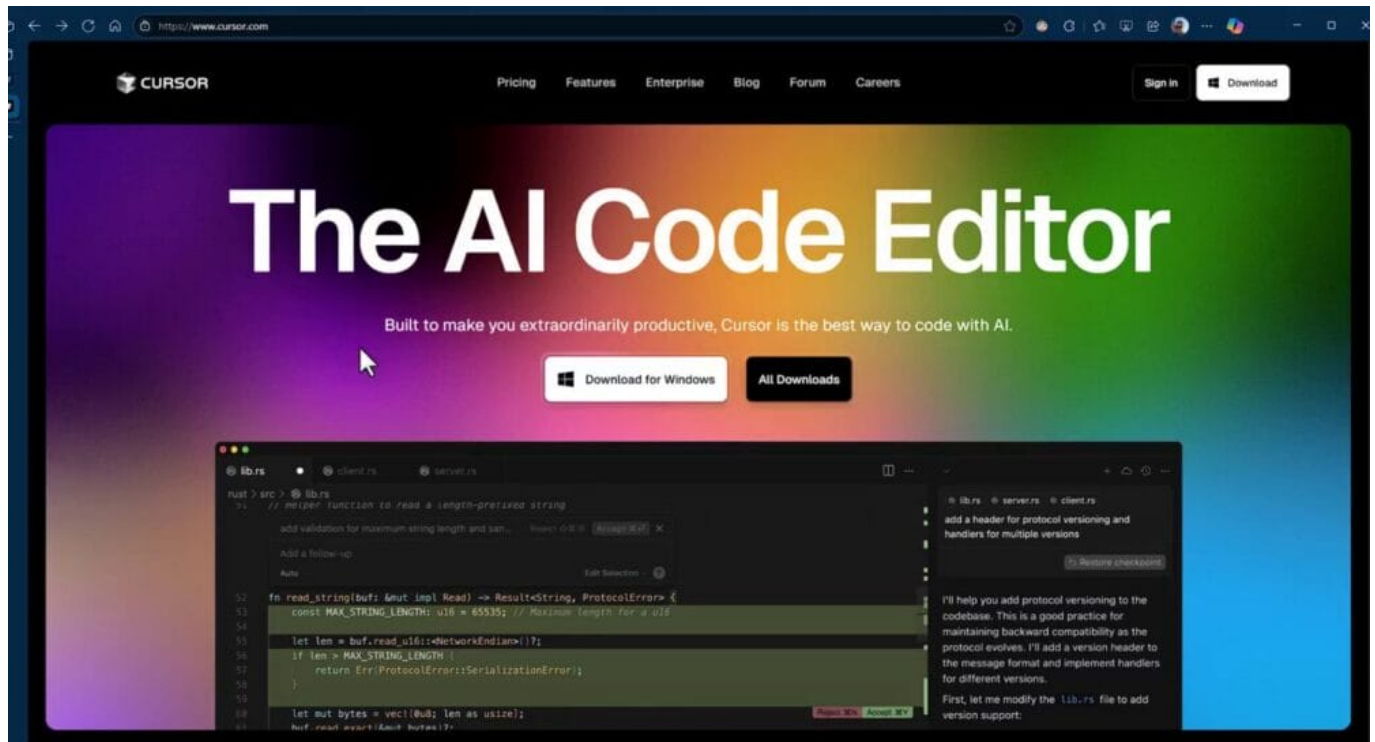
- **Hallucinations:** The AI may import functions or APIs that do not exist. Always cross-check the documentation of any unfamiliar call.
- **Security gaps:** When your project handles user input, for example the image manipulation demo used later in this course, it is easy to introduce exploits. Sanitize MIME types, validate file size, and guard against path traversal.
- **Stale context:** After a massive refactor or a large branch change, reindex the project so that Cursor does not keep answering from an outdated version of your code.
- **Token limits:** Asking the AI to operate on the whole repository at once can easily exceed context limits (upwards of 8,000 tokens). Break tasks into smaller, digestible chunks.

The golden rule of vibe coding is to work in small iterations with rapid tests. Learn the five core features of Cursor, craft context-rich prompts, and follow the vibe loop: plan, regenerate, test, refine, and repeat. Avoid oversized prompts and re-index your code whenever the project changes significantly. Applied consistently, this workflow can multiply your efficiency without sacrificing code quality.

In this lesson, you will set up the development environment used throughout the course by installing the Cursor AI Code Editor, opening a project folder, and getting familiar with the chat panel and its available modes. By the end of the lesson, you will be ready to start building applications with the help of an AI assistant.

Visiting cursor.com

Open your web browser and navigate to cursor.com. The landing page introduces the Cursor AI Code Editor and lets you read about its features before installing it. Spend a minute exploring the page so you know what the tool offers.



Signing in and creating an account

Before downloading the editor, use the Sign In button at the top of the page. If you do not already have an account, create one so that you can access all of the services that Cursor provides. The free plan is enough to build a wide range of applications, and you can always upgrade your subscription plan later if you want to unlock additional capabilities.

Installing the editor

Download the installer for your operating system from the Cursor website, run it, and then sign in with the same account you just created. Using the same account on the website and in the desktop app keeps your settings and subscription linked.

Starting or opening a project

Once Cursor is installed and open, the welcome screen gives you three ways to start working:

- Create a brand new project.
- Open an existing project from your computer.
- Clone a repository from GitHub and open it directly in the editor.

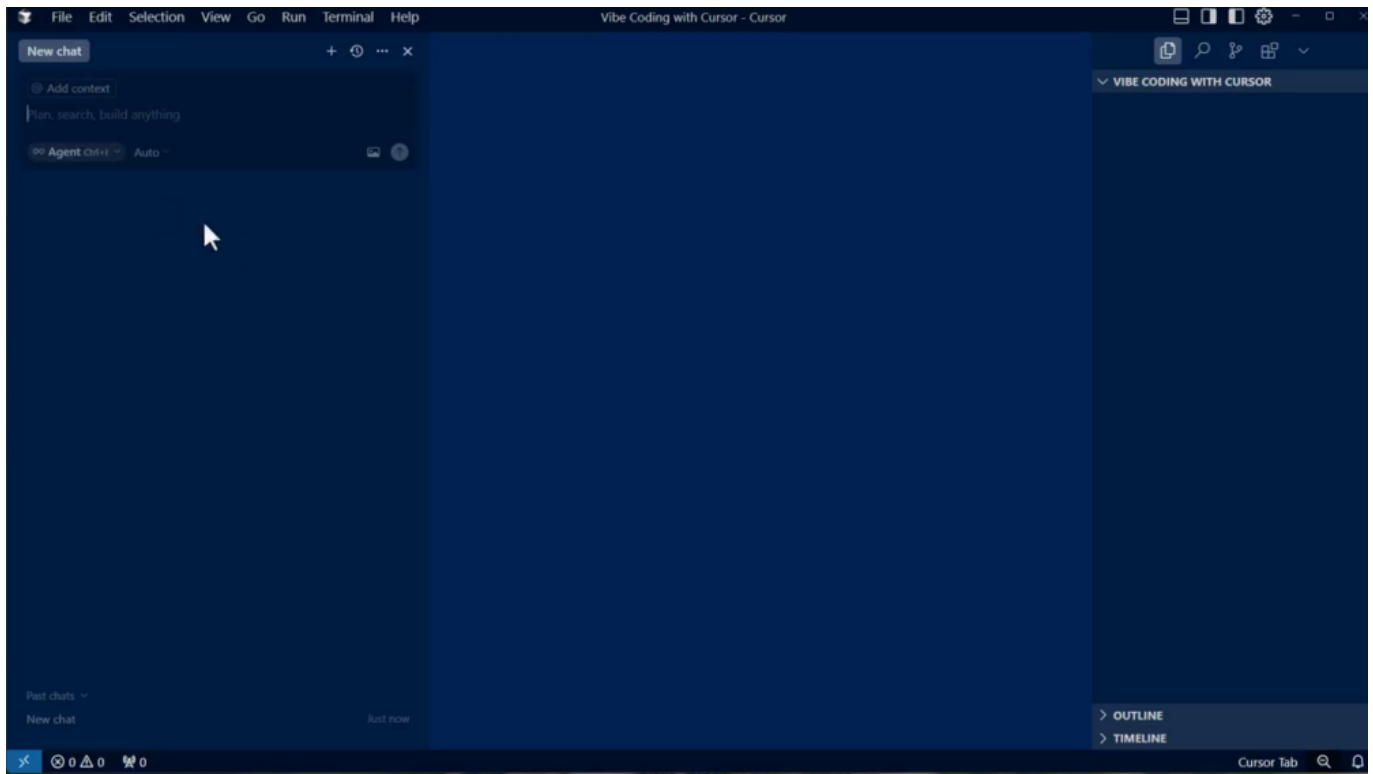


Opening the chat panel

Pressing Ctrl + V opens the Cursor chat panel. If you have not previously customized the VS Code style layout, the panel typically appears on the left side of the window. This panel is where you will interact with the AI assistant throughout the course.

Opening the project folder

To work on a specific project, use the Open Folder option and pick the folder you want to use. In this lesson, we open a folder called Vibe Coding with Cursor from the desktop, which becomes the workspace for the rest of the course. After selecting the folder, Cursor loads it into the Explorer and the chat panel becomes ready for use.



Using the chat panel

The chat panel is where you ask the AI to help you build the project. It offers several useful controls:

- Start a brand new chat session.
- Browse your chat history to revisit earlier conversations.
- Add context such as files, code snippets, documentation, or past chats so the AI can base its answers on the right information.
- Upload an image, which is helpful when you want to describe a visual design or reference a screenshot.

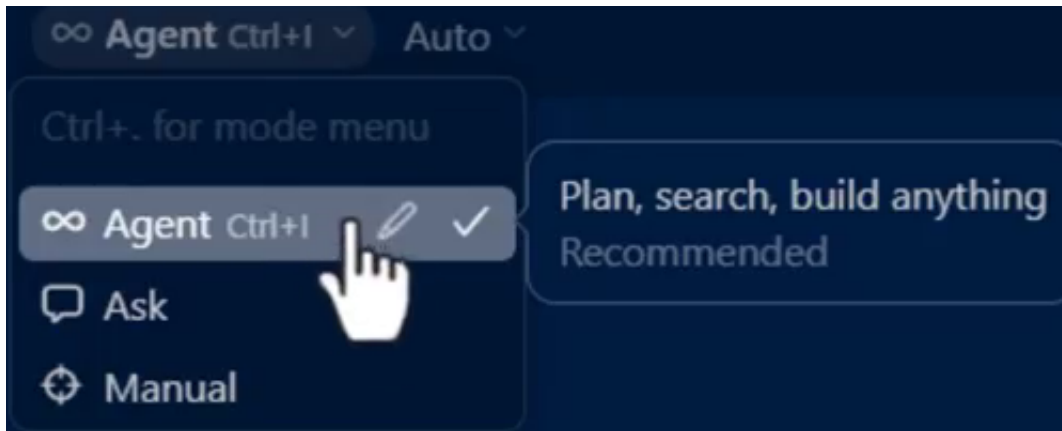
Choosing a chat mode

The most important setting in the chat panel is the mode selector. It controls how much autonomy the AI has when responding to you. Cursor offers three modes:

- **Agent:** with only a small amount of information from you, the AI will plan the work, research what it needs, and build the project for you.



- **Ask:** the AI answers questions about your codebase without making changes to your files.
- **Manual:** you decide exactly what context gets added, and no tools are provided to the AI on your behalf.



If you already know what you want to build, Agent mode is the most convenient choice. If you are unsure or simply want to learn more about an idea before committing to it, Ask mode is the safer starting point.

Trying Ask mode with a sample question

To see the difference in practice, switch the mode selector to Ask. The input field now invites you to ask, learn, or brainstorm rather than to build something end to end.



As an example, we ask how to create an image resizing web app using Flask. Because the chat is in Ask mode, Cursor responds by generating an outline for the project instead of creating any folders or files. This is a useful way to plan a project before you let the AI start writing code.

With Cursor installed, a project folder open, and the chat panel configured, your development environment is ready. In the next lesson, you will use this setup to start building the image resizing web app.



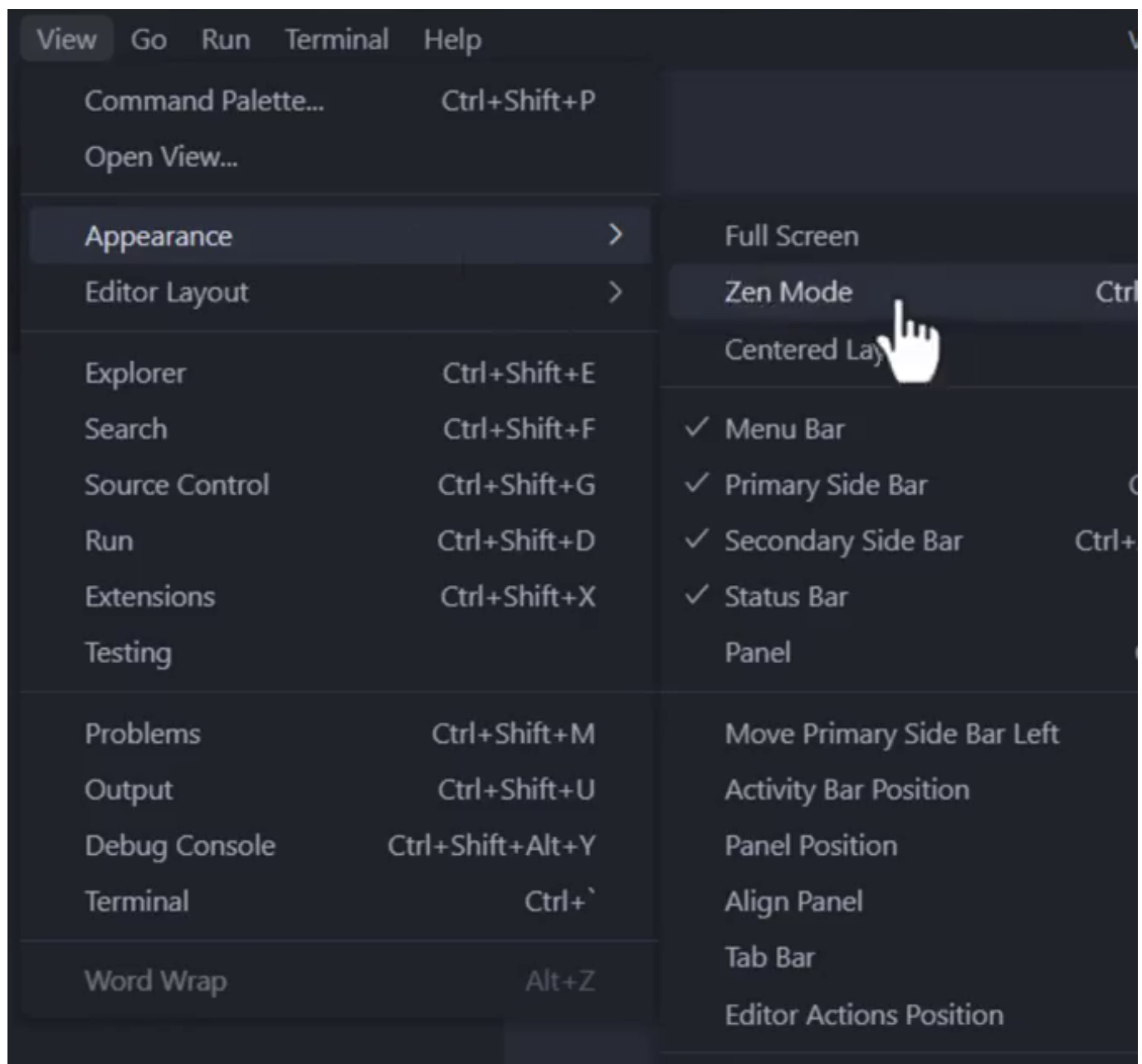
Welcome back. In this lesson, we will use Cursor's AI chat in Agent Mode to scaffold a small Flask web application called **Zenva Resizer**. The app will let users upload images, resize them with Pillow, preview the results in the browser, and download the processed files. By the end of this part, you will have a minimal but working version of the project running locally.

Opening the chat panel in Cursor

Once you have your *vibe coding with cursor* project folder open, you may not see the AI chat panel by default. There are two quick ways to open it:

- Press Ctrl + Alt + B to toggle the secondary side bar that hosts the chat.
- Or use the menu: **View > Appearance** and enable the chat panel from there.

With the panel visible, make sure the chat is set to **Agent** mode so Cursor can create and edit files for you, not just answer questions.





Crafting a clear project prompt

Vibe coding works best when the prompt is specific. Rather than asking for a generic image resizer, we describe the application, the supported file types, the inputs the user must provide, and the technical constraints. Here is the starter prompt used in the lesson:

Build a minimal flask web app called ZenvaResizer that lets users:

1. Upload up to 5 images (PNG, JPEG, JPG)
2. Enter target width, height and quality
3. Preview the resized results in-browser
4. Download images one by one

Use pillow for resizing.

Accept files by extension only for now.

Serve processed images from a temporary "uploads/" folder with Flask's ``send_from_directory``

No authentication, CSRF or header hardening process.

For the UI, use basic Jinja2.

You can use ``secure_filename`` when saving uploads

Let's build!

generate the minimal project, now keep it clean, concise and runnable!

This prompt is a good starting point but it is not the only way to do it. You can also describe the exact endpoints you want (for example, /upload, /download, or just / for the index page), the folder layout, or any other constraint. The more specific you are upfront, the less you will have to clean up later.

Sending the prompt and reviewing generated files

After sending the prompt, Cursor analyzes the request and starts generating files one by one. In the lesson, the agent produced the following files, each of which you accept as it appears:

- requirements.txt: lists the Python dependencies for the project, such as Flask and Pillow.
- app.py: the main Flask application, containing the routes and the backend logic for handling uploads and resizing.
- templates/index.html: the upload form with a small amount of CSS and the inputs for width, height, and quality.
- templates/results.html: the page that displays the resized image preview and the download link.

Once everything is generated, the chat shows a short summary of what was built and how to run it. The features described match the prompt: support for up to five images, customizable width, height, and quality, an in-browser preview, and per-file downloads.

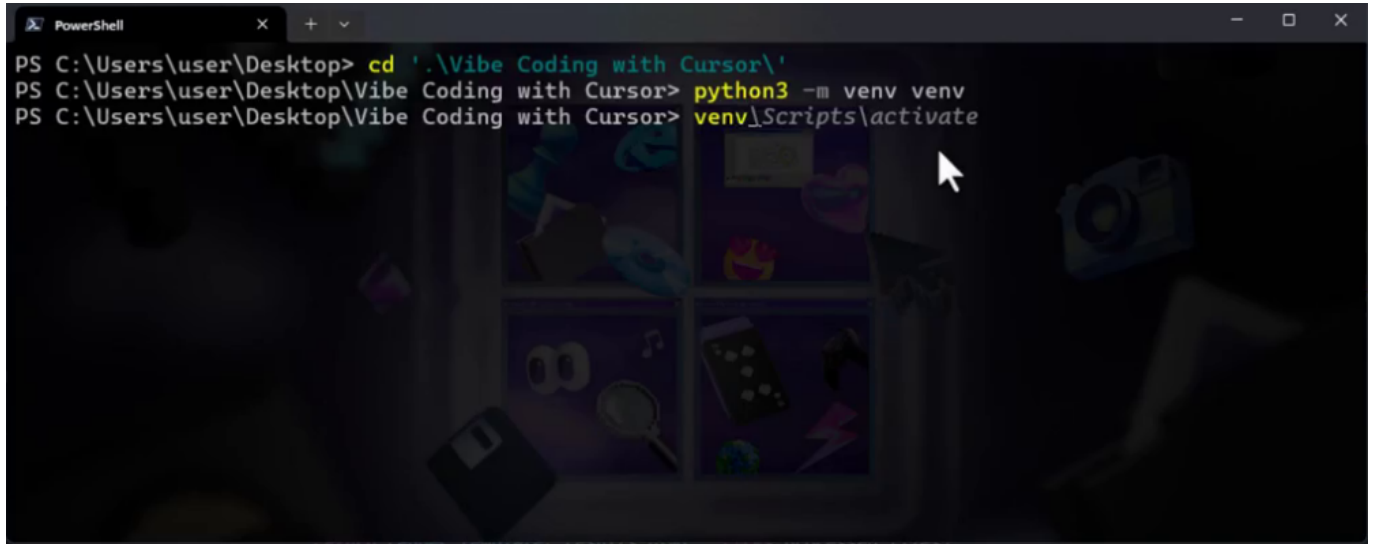
Creating the virtual environment

To run the app, open the integrated terminal and navigate into the project folder. You will then create a Python virtual environment so the project's dependencies stay isolated. We use python3 here because our PowerShell is configured that way; on most machines you can use plain python instead.

```
cd vibe-coding-with-cursor  
python3 -m venv venv
```

Activate the virtual environment. The exact command depends on your operating system:

- On Windows (PowerShell): `venvScriptsactivate`
- On macOS or Linux: `source venv/bin/activate`



Installing the dependencies

With the environment active, install the packages declared by Cursor in `requirements.txt`. This pulls in Flask, Pillow, and any supporting libraries.

```
pip install -r requirements.txt
```

Running the application

Start the Flask development server by running the main script:

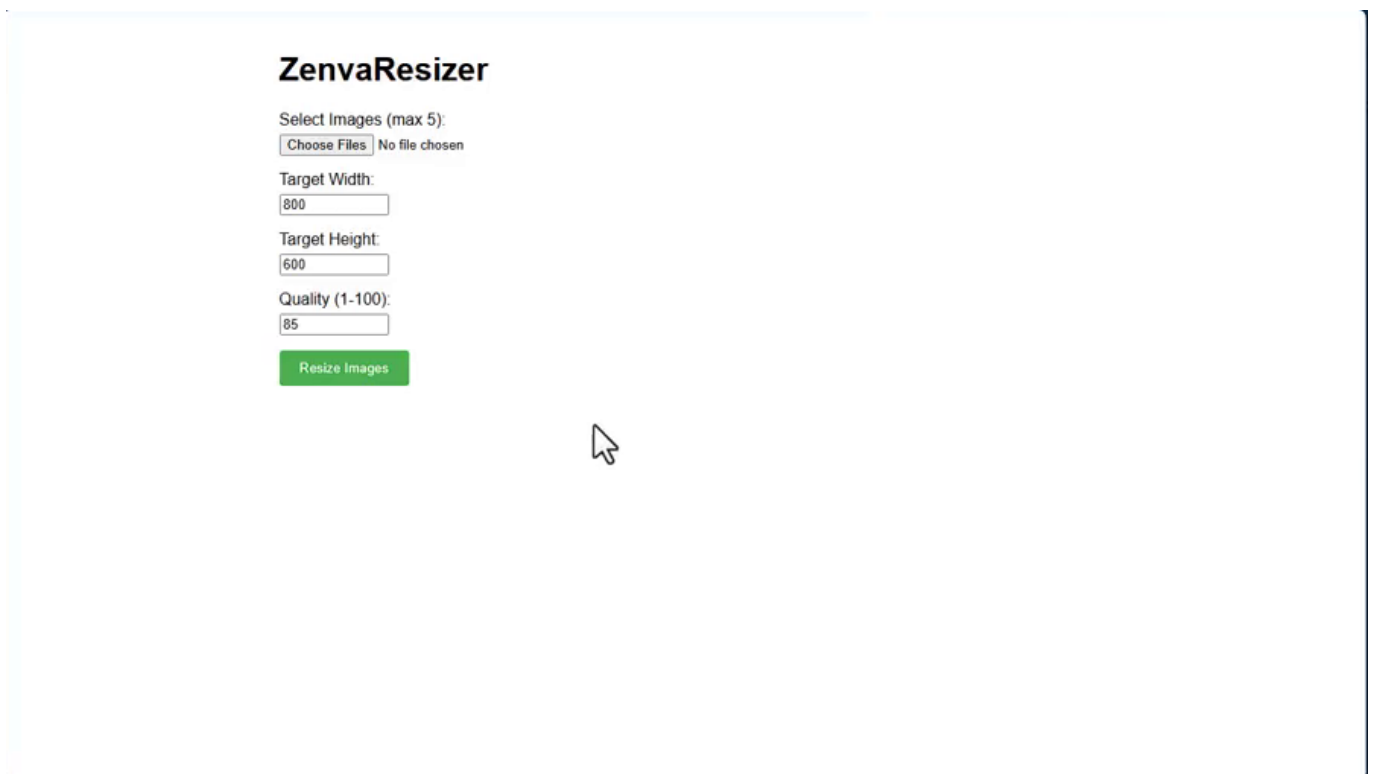
```
python3 app.py
```

Flask will print its startup banner and the local URL the app is listening on, typically `http://127.0.0.1:5000`. You can Ctrl-click that URL in the terminal to open it in your browser.

```
PowerShell
(venv) PS C:\Users\user\Desktop\Vibe Coding with Cursor> python3 app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production
WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 119-392-609
```

Testing the resizer

Once the page loads, you will see a very plain form generated by Jinja2. There is no styling yet, which is intentional for this first pass. The form exposes a file picker, fields for width, height, and quality, and a submit button to process the images.



ZenvaResizer

Select Images (max 5):

No file chosen

Target Width:

Target Height:

Quality (1-100):

To try it out, click **Choose a File** and pick an image. In the lesson, the image is a 124 by 124 PNG. Leave the default width and height (800 and 600), and set the quality to 100 so the resized output keeps as much detail as possible. Then click **Resize Images**.

The results page shows a preview of the resized image. The output looks a bit stretched because the default 800 by 600 dimensions do not match the source image's aspect ratio, which is expected at this stage. Below the preview, the **Download** button serves the new file with its updated name.



Resized Images



If you open the new uploads folder that the app creates next to `app.py`, you will see the processed file there. Inspecting its properties confirms the resize worked: a source image of 124 by 124 has been written out as 800 by 600 pixels.

Wrapping up

At this point, the minimal Zenva Resizer is functional end to end: you can upload an image, resize it with custom dimensions and quality, preview the result in the browser, and download the processed file. In the next lessons, you will iterate on this base by adding UI elements and styling with CSS, and by extending the project with more capable image processing options.

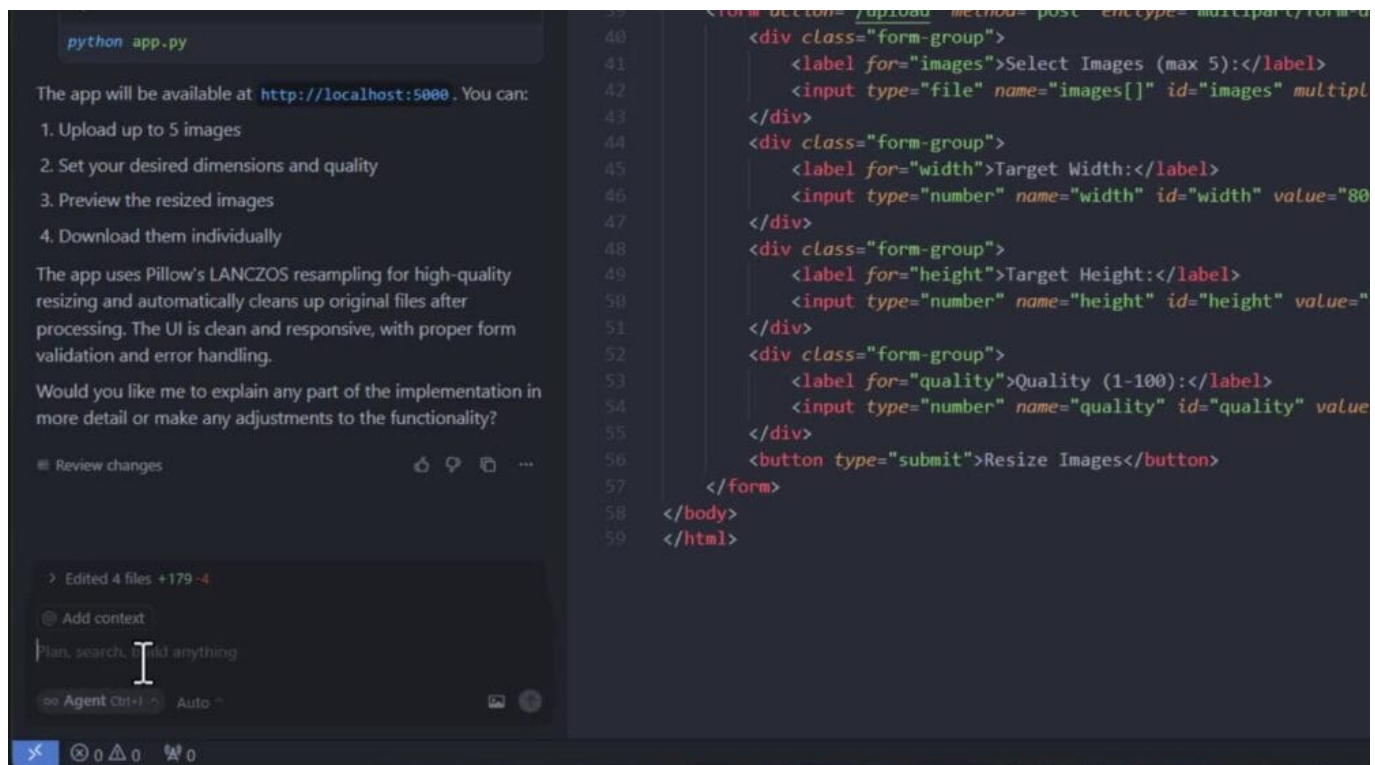
In the previous lesson, the backend of the image resizer was working as expected. In this lesson, the focus shifts to the user interface: using Cursor in agent mode, we will ask the AI to give the web app a modern, neon-themed look and feel, and then extend it with header and footer sections. The goal is to show how a simple prompt can drive a full visual overhaul of a Flask application without hand-writing CSS.

Writing a prompt to upgrade the look and feel

With Cursor still in agent mode, the first step is to describe what we want in plain language. Adding a creative direction, such as a color palette, helps the agent make more confident styling decisions. The prompt used in the lesson is:

Now I want you to upgrade the look and feel of our web app.

Maybe you can use neon colors!



After sending the prompt, Cursor proposes a new CSS code snippet for the `index.html` template, and then does the same for the results page. Each proposed change is reviewed and accepted in turn.

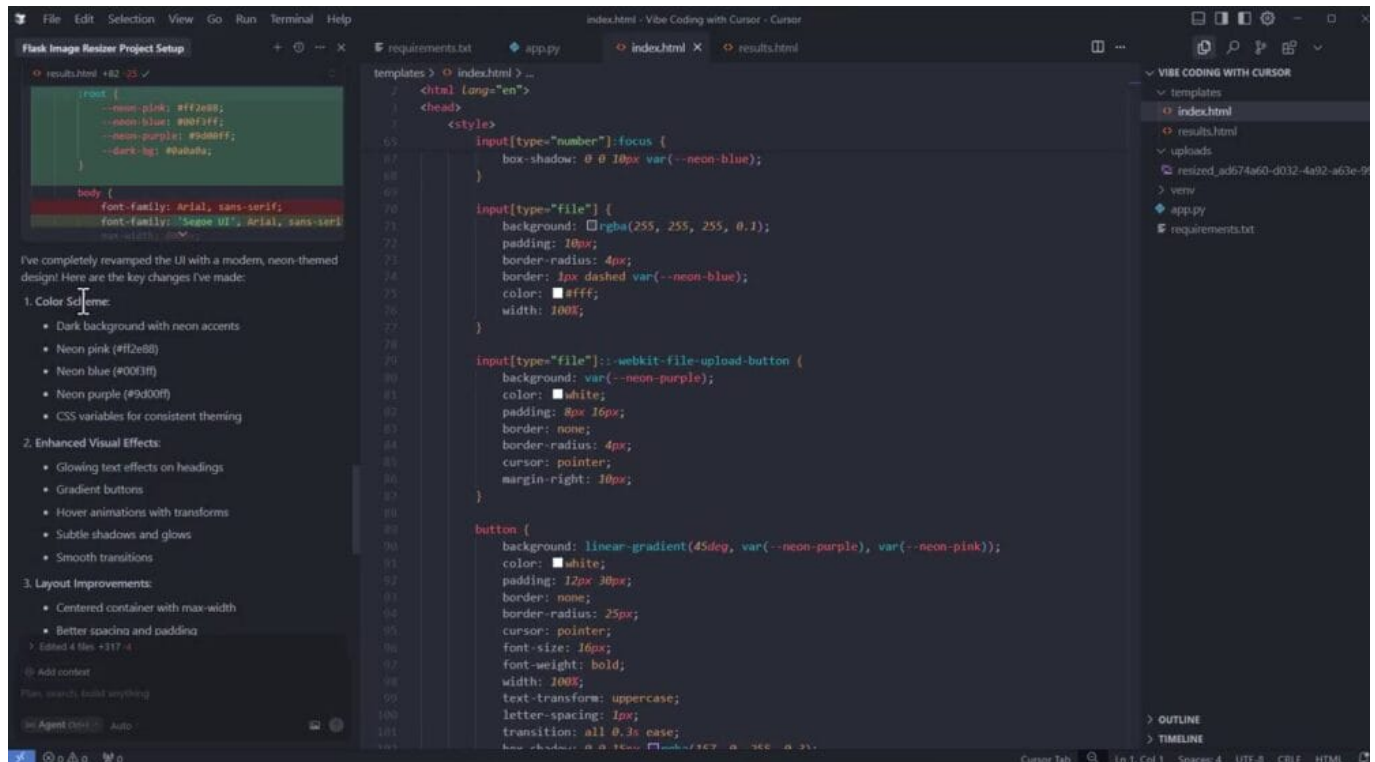
The agent then reports that it has completely revamped the UI with a modern neon-themed design, including:

- A new color scheme built around neon pink, blue, and purple.
- Enhanced visual effects such as glows and shadows.
- Layout improvements for the form and result panels.
- Interactive elements with hover states.
- Typography enhancements.

The heart of the redesign is a small set of CSS custom properties defined at the top of the stylesheet,

which every other rule then references:

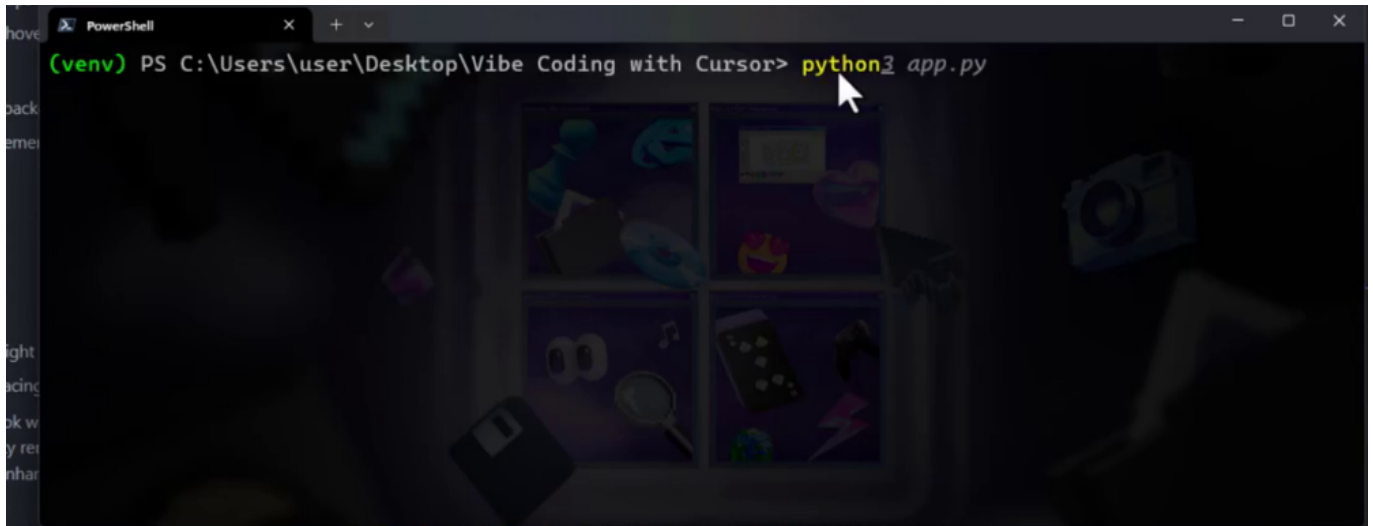
```
:root {  
  --neon-pink: #ff2e88;  
  --neon-blue: #00f3ff;  
  --neon-purple: #9d00ff;  
  --dark-bg: #0a0a0a;  
}
```



Running the application

With the updates accepted, the next step is to run the Flask app locally and see the new theme in action. The virtual environment is already active in the integrated terminal, so the only command needed is:

```
python app.py
```

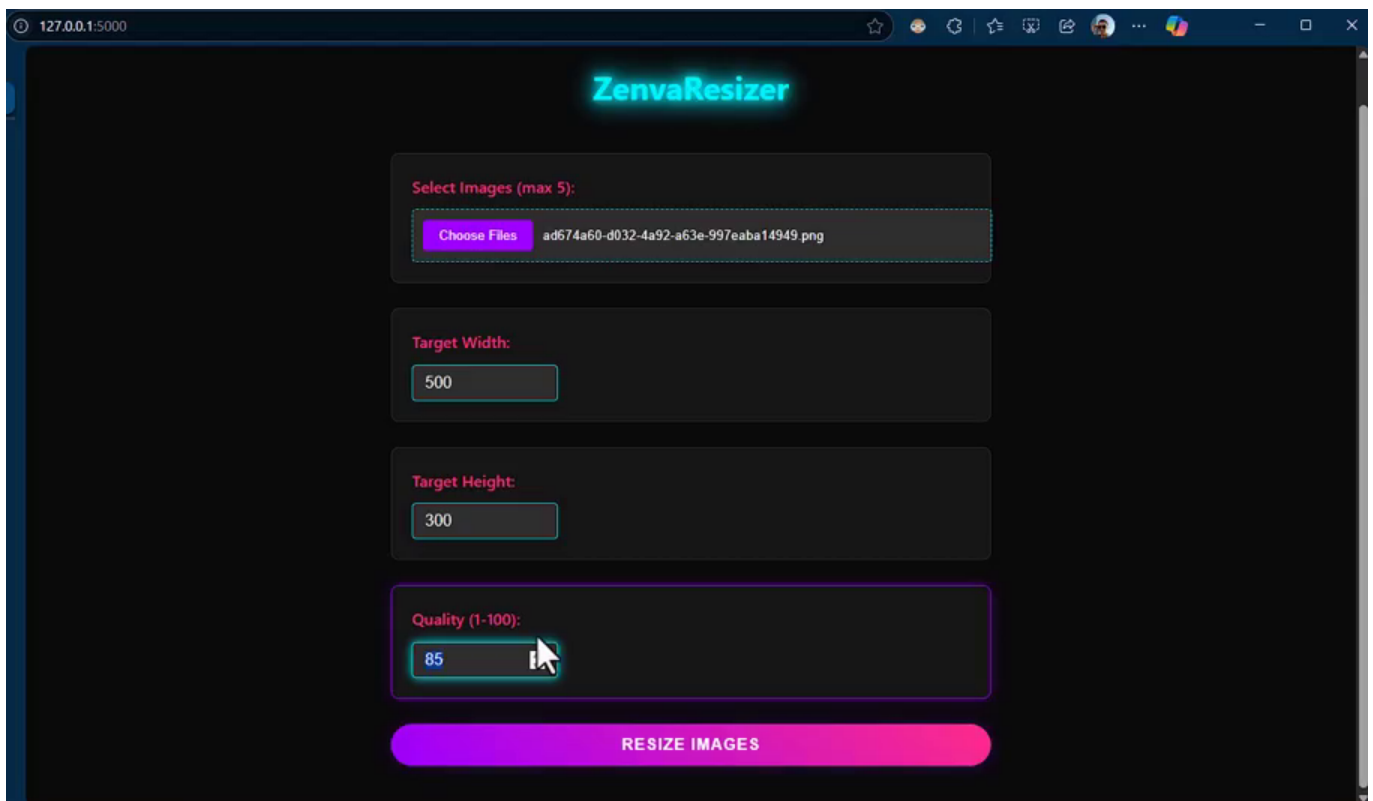


Testing the new design

Opening the browser and navigating to the main page of the app reveals the redesigned upload form. Instead of a plain layout, the page now uses a dark background with neon-accented panels, one for each property: selecting an image, target width, target height, and quality.

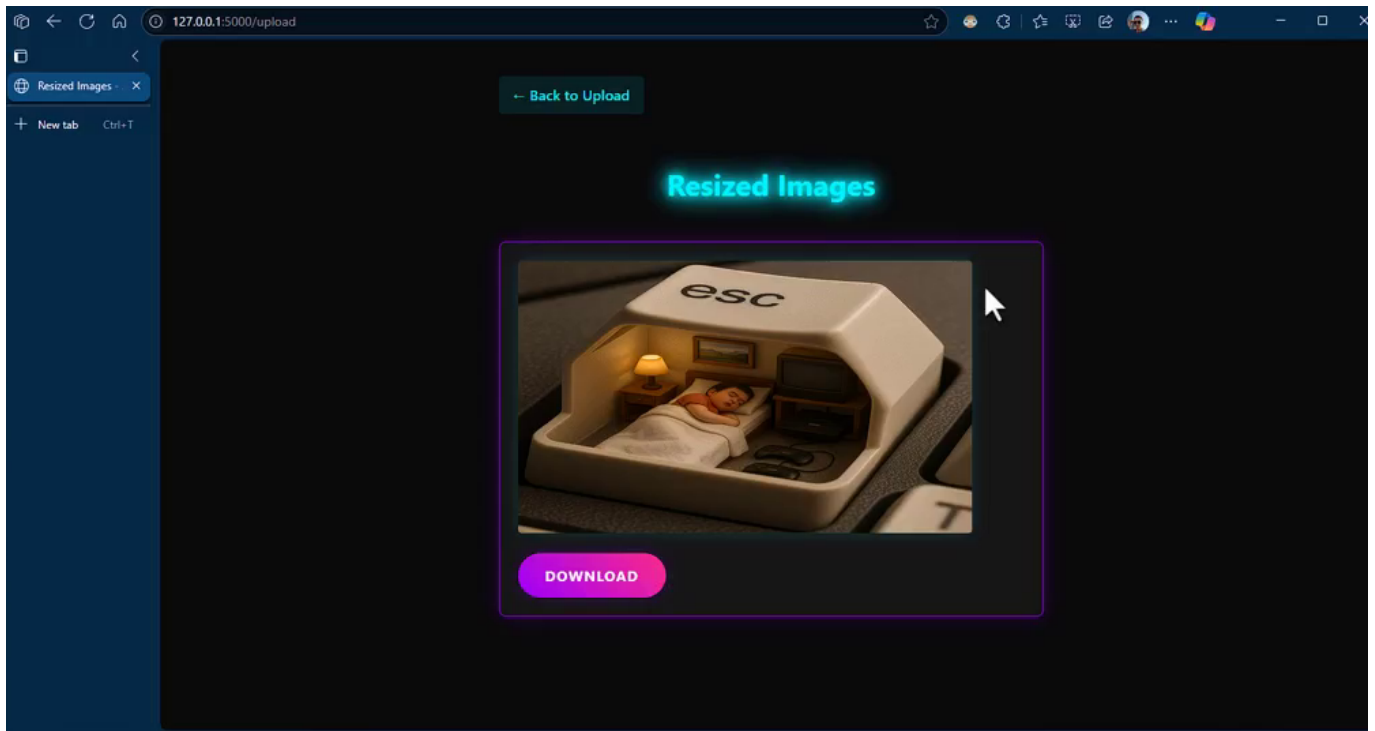
The lesson walks through a quick test:

1. Select an image using the file picker.
2. Set the target width to 500 pixels.
3. Set the target height to 300 pixels.
4. Set the quality to 90.
5. Click **Resize Images**.





After submitting, the preview page also appears with the new neon design. Hovering over a panel triggers a subtle glow effect, a small interactive touch added by the redesign. From here, the resized image can be downloaded directly.

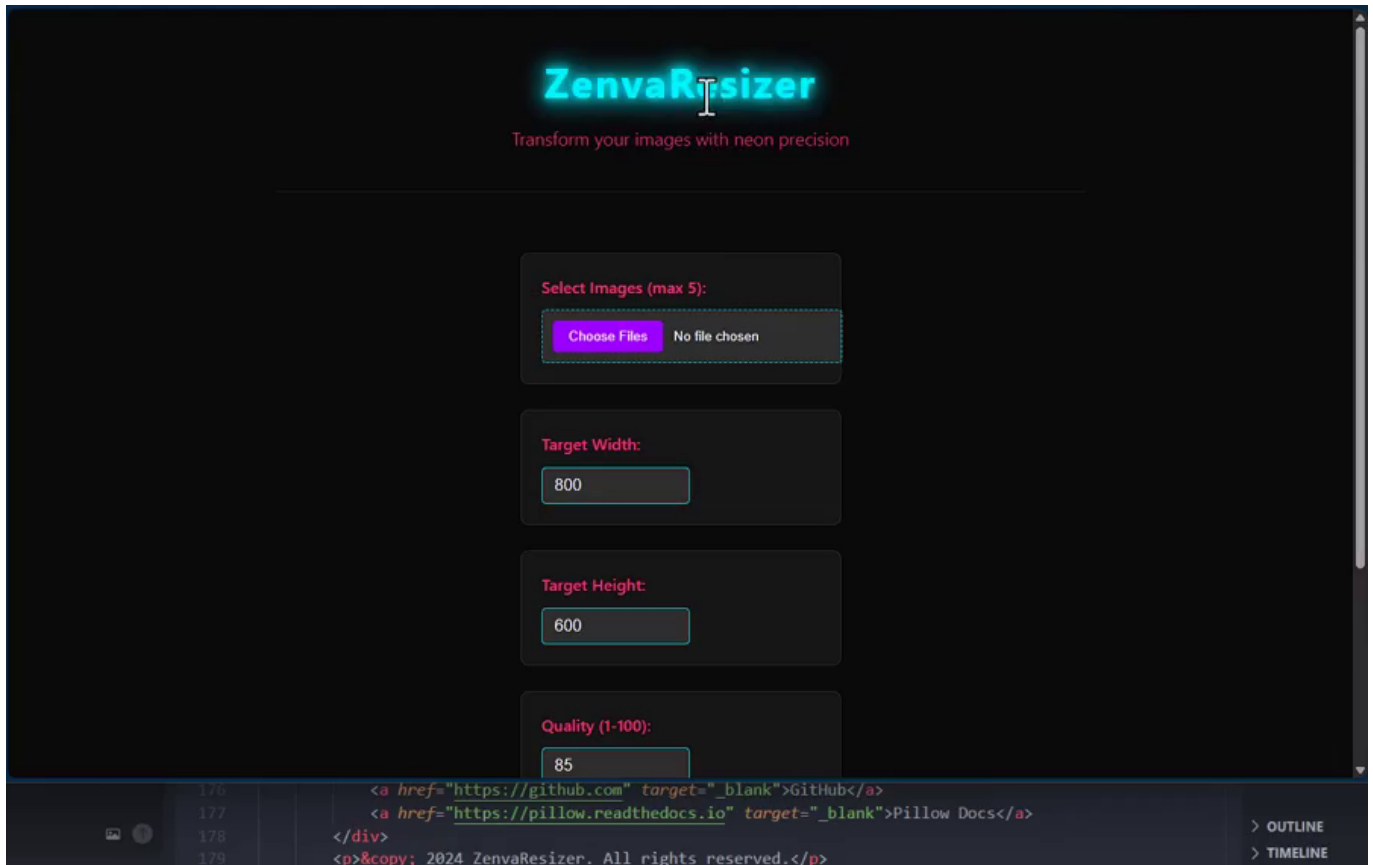


Adding header and footer sections

To round out the layout, the next prompt asks Cursor to add structural elements around the form. The prompt is intentionally short:

Add header and footer sections.

Cursor proposes updates to both `index.html` and `results.html`. Accepting each change inserts a header with the ZenvaResizer logo and a tagline at the top of the page, and a footer with navigation links and copyright information at the bottom. A quick refresh of the browser confirms that the new elements are in place.



One small detail worth noting: the generated footer may include a copyright year that does not match the current year, since language models can be off by a year or two depending on their training data. It is an easy fix, but a useful reminder that AI-generated output should always be reviewed before shipping.

With these updates, the image resizer now has a cohesive, modern, neon-themed design complete with a proper header and footer, all driven by a handful of natural-language prompts.



Welcome back to this course on vibe coding with an AI-powered code editor. At this point our web application can already resize images, but the experience is still quite basic. In this lesson we will discuss a few ideas for improving the project, touch on the importance of security when building with AI assistance, and then use Cursor to add a rotation feature and generate a README for the project.

Ideas for polishing the user experience

Once the core image resizing works, there are several directions you can take the app in to make it more polished and user friendly. A few natural next steps include:

- A drag-and-drop zone that previews the uploaded images before processing.
- An option to download all processed images at once as a zip file, triggered by a “Download All” button.
- Additional image transformations such as rotation, which we will add in this lesson.

A quick word on security

When you build a product with an AI-powered code editor, the editor does not usually think about securing the application on its own, because as human engineers we also tend not to start our projects secured by design. If you are aware of the basic security issues that affect web applications, however, you can always ask the editor to improve the project. Keeping a few fundamental concerns in mind as you go will help you end up with a more robust application:

- Validating user input so the server does not trust data coming from the browser blindly.
- Controlling who can access which parts of the application through proper authentication and authorization.
- Protecting sensitive data both in transit and at rest.

Adding an optional rotate function

For our last feature in this part of the project we will add rotation, which will continue to use the Pillow imaging library under the hood. Instead of writing the code by hand, we can describe the feature in plain English and let Cursor implement it.

Open a new chat in Cursor and send a prompt such as:

Add optional rotate function.

Cursor starts by researching the code base to understand the project structure, reads `app.py`, and then adds the rotation feature. You can see it introduce a new block in the upload route that reads a rotation value from the submitted form, and a conditional inside the image processing loop that applies the rotation before the resize step.

The new form field is read just like the other parameters. For example, the upload handler now pulls a rotation value from the request:

```
rotation = int(request.form.get('rotation', 0))
```

And inside the image processing block, the rotation is applied with Pillow only when it is not zero, before the image is resized:



```
with Image.open(input_path) as img:
    # Apply rotation if specified
    if rotation != 0:
        img = img.rotate(rotation, expand=True, resample=Image.Resampling.BICUBIC)
    # Resize after rotation to maintain aspect ratio
    img = img.resize((width, height), Image.Resampling.LANCZOS)
    img.save(output_path, quality=quality)
```

On the front end, Cursor also adds a matching input to index.html so the user can type a rotation value in degrees:

```
<div class="form-group">
    <label for="rotation">Rotation (degrees)</label>
    <input type="number" id="rotation" name="rotation" value="0" min="0" max="360">
</div>
```

Once the changes look correct, accept all of them so they are saved to the project files.

Testing the rotation in the browser

With the edits applied, open the web browser and refresh the application page. The form now includes a rotation property alongside the existing width, height, and quality fields.



Select Images (max 5):

Choose Files No file chosen

No file chosen

Target Width:

800

Target Height:

600

Quality (1-100):

85

Choose an image, set the rotation to 90 degrees, and press the button to resize the images. After a moment the processed image appears with the new size and rotated orientation, confirming that the rotation option is working end to end. You can try other angles such as 45 or 180 to see how Pillow handles non-right-angle rotations, and note that because `expand=True` is used when rotating, the output canvas grows to fit the rotated content before the final resize step.

Generating a README with Cursor

To wrap up this part of the project, we can ask Cursor to create a README file so that anyone who opens the repository can understand what the project does and how to run it. Send a prompt along the lines of:

Now please finalize and create us a `readme.md` file!

Cursor generates a new `README.md` with an overview of the project, a feature list, requirements, installation steps, and usage instructions. Review the generated file and accept the changes to add it to the project.



```
README.md > # ZenvaResizer
```

```
# ZenvaResizer
```

```
## Features
```

- 🎨 ****Quality Control****: Adjust output image quality (1-100)
- 🌈 ****Modern UI****: Sleek neon-themed interface with smooth animations
- 🚀 ****Fast Processing****: Efficient image processing using Pillow
- 📱 ****Responsive Design****: Works on both desktop and mobile devices

```
## Requirements
```

- Python 3.7+
- Flask
- Pillow (PIL)
- Werkzeug

```
## Installation
```

```
1. Clone the repository:
```

```
```bash
```

```
git clone https://github.com/yourusername/zenvaresizer.git
```

```
cd zenvaresizer
```

```
```
```

```
2. Create and activate a virtual environment:
```

```
```bash
```

```
python -m venv venv
```

```
source venv/bin/activate # On Windows: venv\Scripts\activate
```

```
```
```

```
3. Install dependencies:
```

```
```bash
```

## What's next

At this stage our image resizer can upload images, resize them, and rotate them, and the project is documented with a README. In the next lesson we will continue building on top of this foundation and explore what else we can achieve by collaborating with an AI-powered code editor.



Over the previous lessons, we built a simple image resizer together. The app is clean, useful, and functional, and we even spiced it up with some style and added resolution features. Vibe coding, however, is all about the flow: taking simple ideas and making them your own. This lesson hands the project over to you and sets a short challenge to push it further.

### **Your challenge**

Take the image resizer app we built and make it vibe with your own twist. You are free to pick any direction you like, but here are a few starting points to spark ideas:

- Add a filter, such as grayscale or a blur effect, using the Pillow Imaging Library.
- Let users drag and drop their image straight into the web app.
- Create a preview panel that shows live image updates as settings change.

The goal here is not to follow the tutorial step by step. It is to own the web, push the limit just a little, and remix, restyle, and reimagine what we built.

### **Bonus challenge**

If you want to push things even further, try describing your new idea in a single prompt and let the Cursor agent build it for you. This is a great way to practice writing clear, focused prompts and to see how far you can get by trusting the agent with a well-described goal.

Good luck, and have fun making this project your own.



In this lesson of the Vibe Coding Image Resizer series, the ZenvaResizer web application is extended with a powerful new capability: watermarks. Watermarking is one of the most common image editing operations on the web, useful for protecting digital artwork, keeping branding visible on every exported photo, or gating premium features behind a paywall. Rather than writing every line by hand, the feature is designed and built through a guided conversation with Cursor's AI, using Ask Mode to plan and Agent Mode to apply concrete changes to the Flask project.

## Why watermarks?

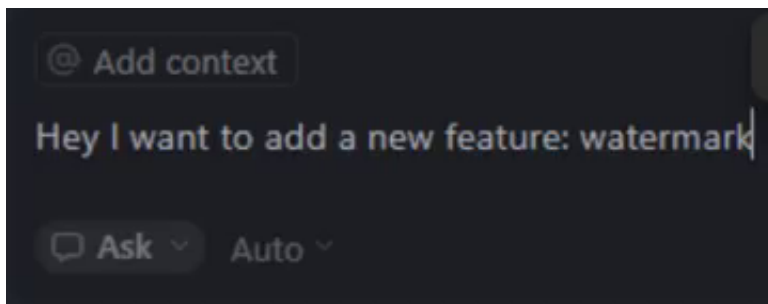
Watermarking appears in nearly every image editing tool in some form. It can be a subtle brand on a portfolio image, a bold "SAMPLE" overlay on a preview, or a configurable stamp that lets end users label their own output. Adding this feature to ZenvaResizer is a good exercise because it touches both the backend image processing pipeline and the frontend form, so it is a realistic test of how well AI pair programming scales beyond small changes.

## Brainstorming in Ask Mode

Before asking Cursor to edit any files, the chat panel is switched from Agent Mode to Ask Mode. Ask Mode is useful for planning: the AI can reason about the feature, suggest properties, and sketch an implementation plan without modifying the workspace. The first prompt is intentionally open-ended:

```
Hey I want to add a new feature: watermarks, what do you think?
```

```
can you create a plan for this, like properties, or ability to add custom texts written by the user???
```

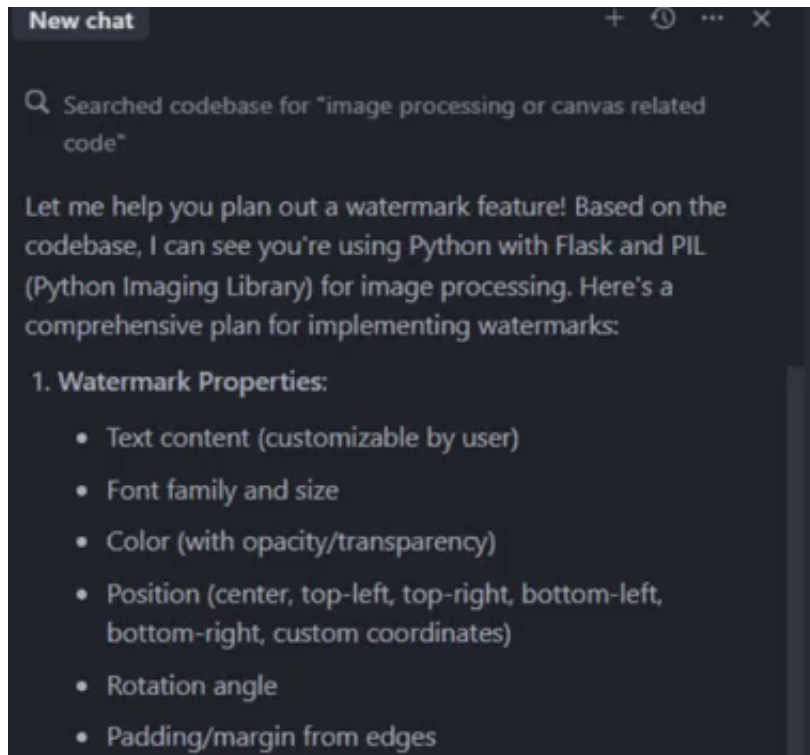


## Reviewing the plan

Cursor responds with a structured plan. It identifies the properties the watermark system needs to handle when drawing text on top of an image, along with an outline of how the backend function might look. The properties it proposes include:

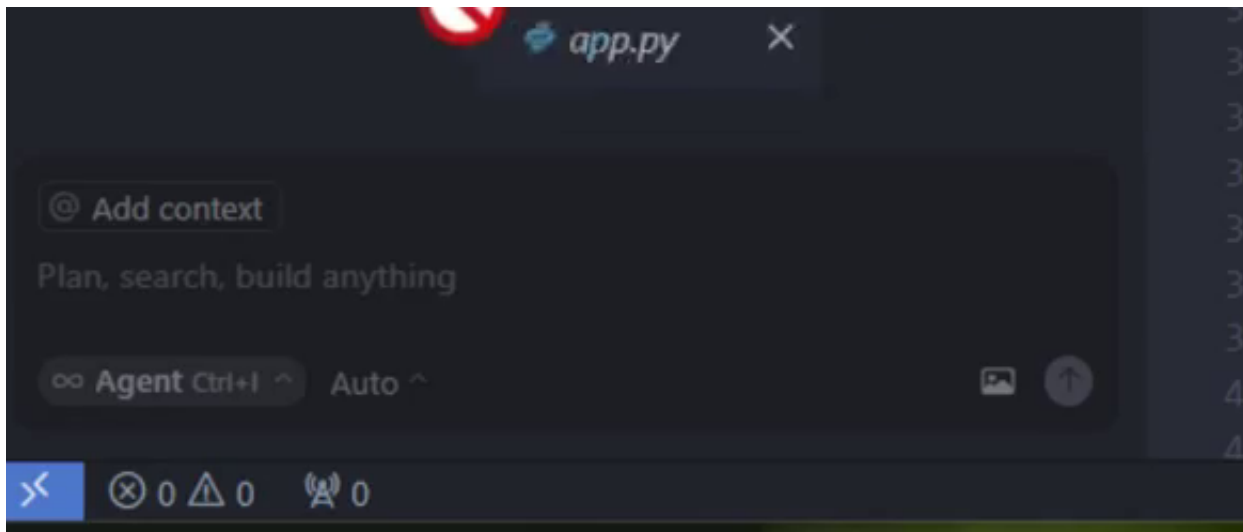
- Text content, customizable by the user
- Font family and size, possibly using Google Fonts
- Color, with an opacity or transparency value
- Position (center, corners, or custom coordinates)
- Rotation angle
- Padding or margin from the edges

Alongside the property list, the response also includes a suggested implementation that creates a transparent layer, loads a font, calculates the text size and position, and draws the watermark onto the layer before compositing it over the image. Frontend UI elements and a few security and performance tips round out the plan.



### Switching to Agent Mode and adding context

Not every part of the plan is needed. The most valuable section is the list of watermark properties, which can be reused as a precise spec for the next prompt. The mode selector at the bottom of the chat is switched back to Agent Mode, so Cursor can actually edit files. To make sure the AI knows which file to update, `app.py` is dragged into the chat as context.



The follow-up prompt combines the property list with a couple of extra directives: suggest using Google Fonts for the font family, and provide a text field so users can supply custom watermark text, falling back to ZenvaResizer when they do not:

I want to add a new feature: watermark, these are properties that we need to handle:

Watermark Properties:

Text content (customizable by user)



Font family and size

Color (with opacity/transparency)

Position (center, top-left, top-right, bottom-left, bottom-right, custom coordinates)

Rotation angle

Padding/margin from edges

For the font family and size properties, maybe you can use Google fonts.

Also, there should be one textfield to add custom text by the user for the watermark operation. So if the user does not give any kind of water text, you can put ZenvaResizer as a watermark.

## Generated backend code

With Agent Mode and the file context in place, Cursor edits app.py directly. The first change is a small configuration class that bundles all the watermark properties together with sensible defaults, so the rest of the code can pass a single object around:

```
class WatermarkConfig:
 def __init__(self, text="ZenvaResizer", font_size=36,
 color=(255, 255, 255), opacity=128, position="center",
 rotation=0, padding=20):
 self.text = text
 self.font_size = font_size
 self.color = color
 self.opacity = opacity
 self.position = position
 self.rotation = rotation
 self.padding = padding
```

Next comes the drawing function. It creates a transparent RGBA layer the same size as the input image, loads a font, measures the text, computes the correct coordinates based on the requested position, draws the text with the configured color and opacity, optionally rotates the layer, and composites it onto the original image:

```
def apply_watermark(image, config):
 """Apply watermark to the image"""
 # Create a transparent layer for the watermark
 watermark = Image.new('RGBA', image.size, (0, 0, 0, 0))
 draw = ImageDraw.Draw(watermark)

 # Load default font
 try:
 font = ImageFont.truetype("arial.ttf", config.font_size)
 except:
 font = ImageFont.load_default()

 # Calculate text size
 text_bbox = draw.textbbox((0, 0), config.text, font=font)
 text_width = text_bbox[2] - text_bbox[0]
 text_height = text_bbox[3] - text_bbox[1]
```



```
Calculate position based on config
if config.position == "center":
 x = (image.size[0] - text_width) // 2
 y = (image.size[1] - text_height) // 2
elif config.position == "top-left":
 x = config.padding
 y = config.padding
elif config.position == "top-right":
 x = image.size[0] - text_width - config.padding
 y = config.padding
elif config.position == "bottom-left":
 x = config.padding
 y = image.size[1] - text_height - config.padding
elif config.position == "bottom-right":
 x = image.size[0] - text_width - config.padding
 y = image.size[1] - text_height - config.padding

Draw text
draw.text((x, y), config.text, font=font, fill=(*config.color, config.opacity))

Rotate watermark if needed
if config.rotation != 0:
 watermark = watermark.rotate(config.rotation, expand=True)
 # Recenter the rotated watermark
 x = (image.size[0] - watermark.size[0]) // 2
 y = (image.size[1] - watermark.size[1]) // 2
 watermark = watermark.crop((0, 0, image.size[0], image.size[1]))

Convert image to RGBA if it isn't already
if image.mode != 'RGBA':
 image = image.convert('RGBA')

Composite watermark onto image
return Image.alpha_composite(image, watermark)
```

The upload route is also updated so it reads the new watermark fields out of the submitted form, builds a WatermarkConfig, and applies the watermark after the resize step:

```
Get watermark parameters
watermark_text = request.form.get('watermark_text', 'ZenvaResizer')
font_size = int(request.form.get('font_size', 36))
color = tuple(map(int, request.form.get('color', '255,255,255').split(',')))
opacity = int(request.form.get('opacity', 128))
position = request.form.get('position', 'center')
watermark_rotation = int(request.form.get('watermark_rotation', 0))
padding = int(request.form.get('padding', 20))

Create watermark configuration
watermark_config = WatermarkConfig(
 text=watermark_text,
 font_size=font_size,
 color=color,
```



```
 opacity=opacity,
 position=position,
 rotation=watermark_rotation,
 padding=padding
)
```

Inside the per-file processing loop, the watermark is applied right after the image is resized, and before it is saved:

```
Apply watermark
img = apply_watermark(img, watermark_config)

Save the processed image
img.save(output_path, quality=quality)
```

## Iterating: dropping Google Fonts

In its summary, Cursor notes that the initial implementation uses the Google Fonts API and adds requests to the requirements file so it can download fonts. In practice, using the Google Fonts API means signing up for an API key and managing a quota, which is unnecessary complexity for a small project. This is a good moment to iterate. A second prompt tells Cursor to remove that integration and use default system fonts instead, and to update the frontend at the same time:

First, remove the Google Fonts, and use default fonts.

Then, update the frontend according to the new changes.

Agent Mode works through both files. In `app.py`, the Google Fonts code is stripped out and the font loader falls back to `arial.ttf`, with `ImageFont.load_default()` as a safety net when Arial is not available. This is exactly the `try / except` branch shown earlier in `apply_watermark`.

## Updated frontend form

On the frontend, Cursor updates `index.html` to add a dedicated Watermark Settings section with its own controls. The new inputs mirror the `WatermarkConfig` fields, so every property the backend accepts is reachable from the UI:

- A text input for the watermark text, with a placeholder that prompts the user
- A number input for font size
- A color picker for the watermark color
- A range slider for opacity, paired with a small display that shows the current numeric value
- A position dropdown with options for center, top-left, top-right, bottom-left, and bottom-right
- A number input for watermark rotation
- A number input for padding

Form submission is rewired to use AJAX: a small JavaScript handler sends the form data with `fetch`, adds the `X-Requested-With: XMLHttpRequest` header so the backend returns JSON, and renders the processed images inline rather than redirecting to a separate results page. Cursor's response also mentions adding Bootstrap to the frontend during this pass for a cleaner layout.

- Opacity slider with live value display
  - Position dropdown
  - Watermark rotation input
  - Padding input
4. Added AJAX form submission with preview of processed images
  5. Added download buttons for processed images
- The watermark feature now works with these default values if not specified:
- Text: "ZenvaResizer"
  - Font: System default (Arial if available)
  - Size: 36px
  - Color: White
  - Opacity: 50% (128)
  - Position: Center

## Defaults and trying it out

Sensible defaults are key to keeping the form friendly: if the user leaves a field blank, the backend falls back to the WatermarkConfig defaults (text ZenvaResizer, font size 36, white color, opacity 128, centered position, no rotation, 20 pixels of padding). That means a visitor can upload an image, ignore the new section entirely, and still get a watermarked result.

After accepting Cursor's edits, Cursor offers follow-up suggestions such as adding a live preview or more customization options. Those are left for later. The lesson ends by opening a terminal and running the Flask app to see the new watermarking pipeline in action.

## Takeaways

This lesson shows how Ask Mode and Agent Mode complement each other when building a non-trivial feature. Ask Mode is used to sketch out the shape of the feature and identify the exact properties that matter. Agent Mode is then used with a focused prompt and explicit file context to turn that spec into working code. Iteration remains important: the first pass uses Google Fonts, and a second



prompt trims that back to system fonts to keep the project lightweight. A good prompt, paired with a clear plan, covers a lot of ground that would otherwise be hand-written boilerplate.

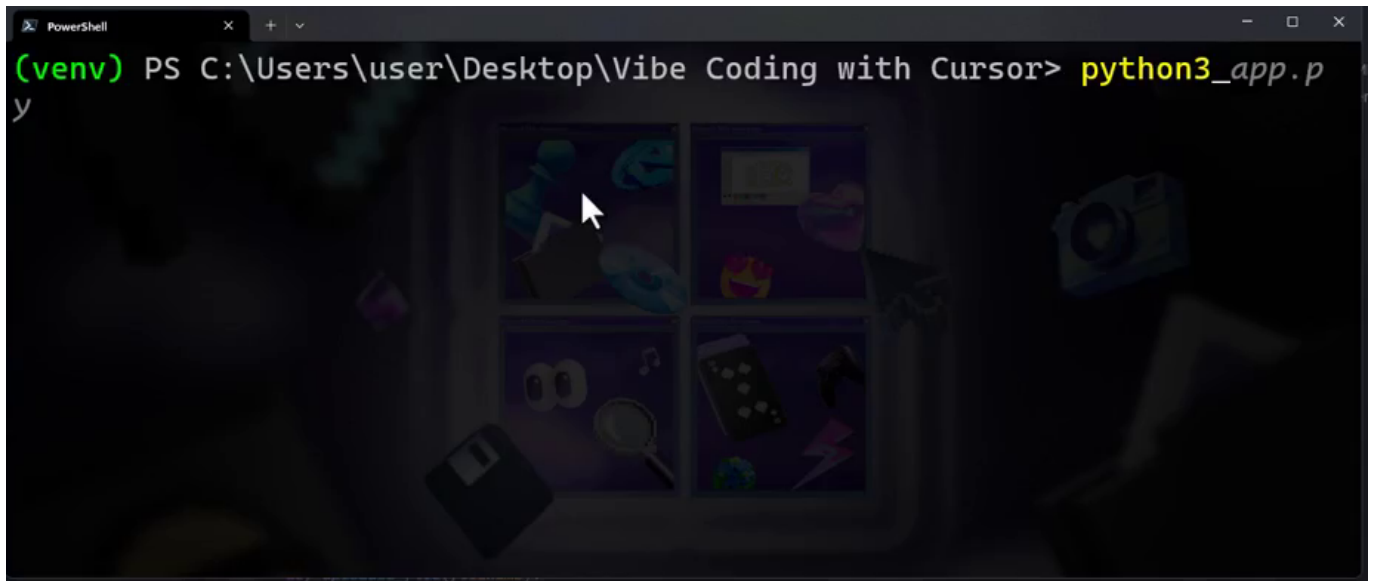
In this part of the Vibe Coding Image Resizer project, we bring the web app online, catch a styling inconsistency left over from an earlier prompt, use Cursor to unify the theme, and then take the watermark feature for a test run end to end. We wrap up with a tiny content tweak and a few ideas for continuing the project on your own.

## Launching the Flask app from PowerShell

With PowerShell open, the first job is to get into the project folder, activate the virtual environment, and start the Flask development server. The commands are the same ones used in earlier lessons:

```
cd desktop
cd "Vibe Coding with Cursor"
venvScriptsactivate
python app.py
```

Once the last command runs, the Flask development server comes up and starts serving the application. Because Flask is running in debug mode, there is no need to restart it after every edit: saving a source file will make the server reload automatically, which keeps the workflow fast during iteration.



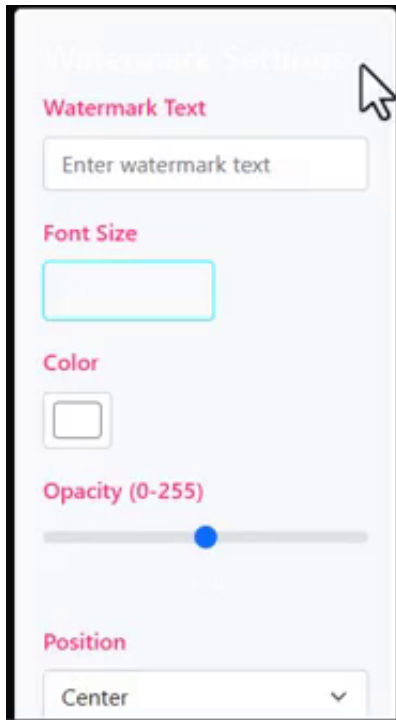
Open a browser and navigate to the local server to see the app:

`http://127.0.0.1:5000`

## Spotting the theme mismatch

The homepage loads, but the Watermark Settings block has a visible problem: its labels are rendering in white, which is almost invisible against the dark neon background of the rest of the page. This is the lingering side effect of an earlier prompt that applied Bootstrap styling only to the watermark config while the rest of the interface used the custom neon CSS. Rather than push on with testing, it is worth fixing the inconsistency first so everything shares one look and feel.





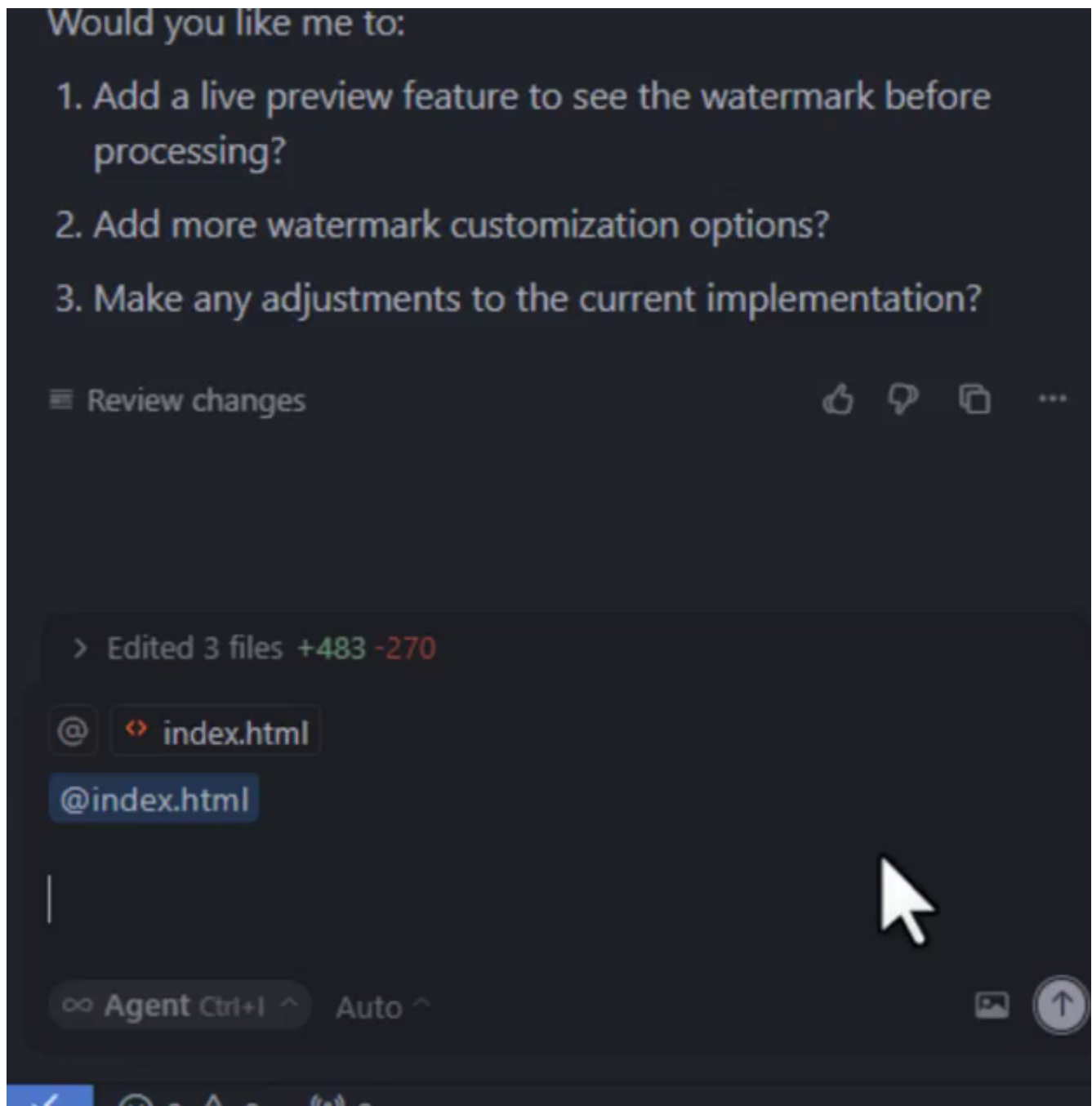
A screenshot of a 'Watermark Settings' panel. It features a title 'Watermark Settings' at the top. Below it are four sections: 'Watermark Text' with a text input field containing 'Enter watermark text'; 'Font Size' with a text input field; 'Color' with a color selection box; and 'Opacity (0-255)' with a horizontal slider bar. At the bottom is a 'Position' dropdown menu currently set to 'Center'. A mouse cursor is pointing at the top right corner of the panel.

### Prompting Cursor to unify the theme

Switching back into the Cursor IDE, the next step is to compose a new prompt in the chat panel. To give the model the right scope, attach index.html as context so Cursor can see the markup and inline styles it needs to change. Then send a clear instruction describing the desired end state:

If you observe the source code, we are using a neon theme for the web app, and you applied Bootstrap for the watermark config part, so there is a mismatch with the frontend.

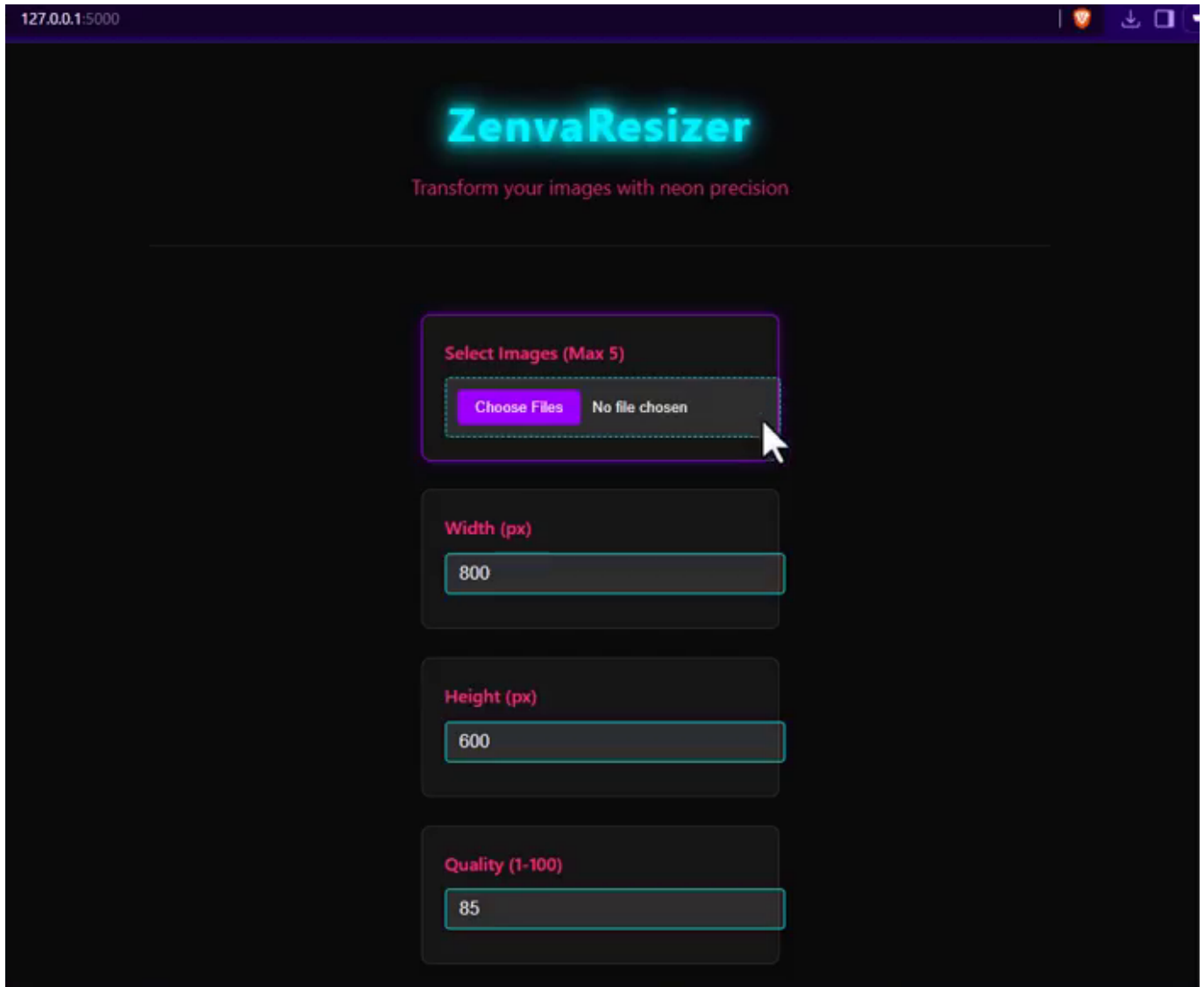
Let's just use our custom neon theme for the entire web page.



After sending, Cursor proposes the edits: removing the Bootstrap classes from the watermark configuration block and extending the existing neon styles to cover those elements. Accept all the changes. The assistant may offer to add extra theme-related effects or animations; for this lesson, decline them and keep the scope focused on the styling fix.

### Verifying the fix in the browser

Back in the browser, a quick refresh is enough to confirm the change. The Watermark Settings section now follows the same neon pink labels and neon blue borders as the rest of the form, and the overall layout feels consistent end to end. One small margin issue is still visible on one of the text fields, but it is minor and will not get in the way of testing the feature.



## Testing the watermarking feature

With the theme unified, it is time to exercise the image processing pipeline. Pick a sample image with the file picker, then fill in the resize and watermark options. A reasonable set of values for a test run:

- Width: 100
- Height: 600
- Quality: 85
- Rotation: 0
- Watermark text: ZenvaResizer
- Font size: 50
- Color: a dark green from the color picker
- Opacity: 128
- Position: Bottom Right
- Watermark rotation: 0
- Padding: 40



Position

Center

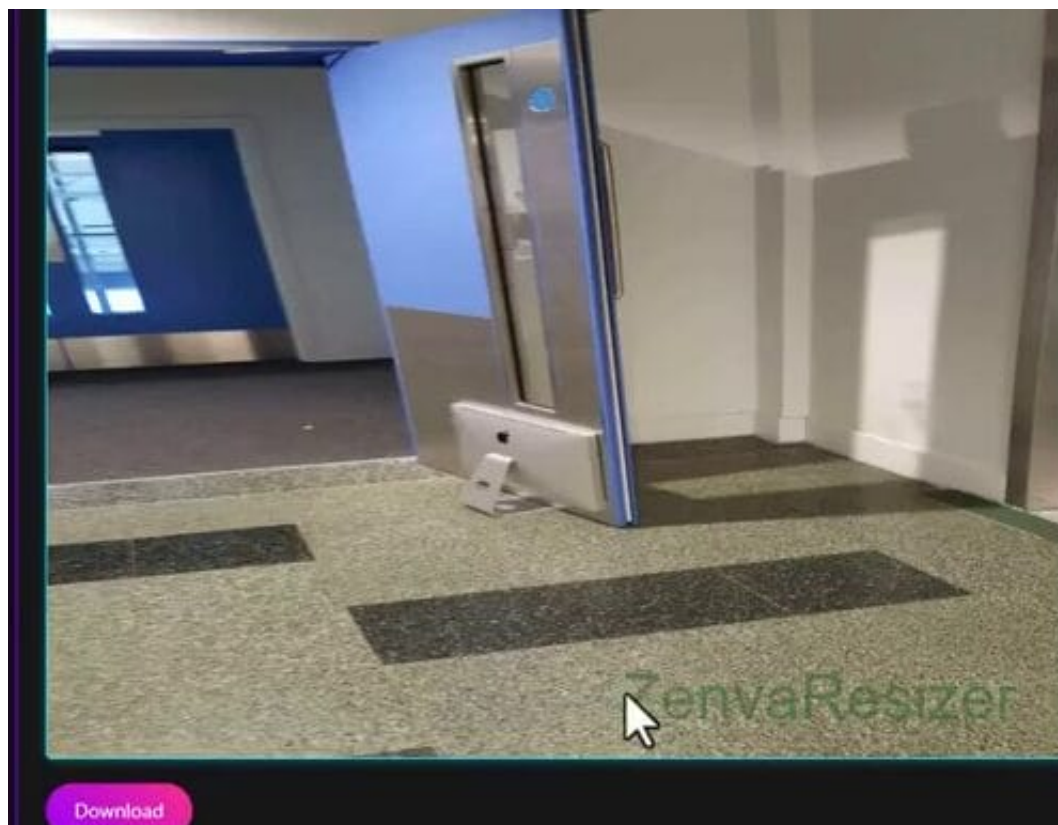
Top Right

Padding (px)

20

PROCESS IMAGES

Click **Process Images**. The Flask backend receives the upload, resizes the image with Pillow, applies the watermark according to the submitted configuration, and returns the processed file. The page updates to show the processed image along with a Download button. The ZenvaResizer text is visible in the bottom-right corner of the output, confirming that the watermarking path is working.





### A small content fix: updating the footer year

Downloading the processed image and opening it locally is a good moment to spot-check the result. While reviewing the page, a small detail stands out in the footer: the copyright year still reads 2024. Open `index.html`, find the footer line, change the year, and save:

```
<p>© 2025 ZenvaResizer. All rights reserved.</p>
```

Refresh the browser and the footer now shows the correct year. This is a tiny change, but it is a good illustration of how quickly content tweaks can be applied when the app is already up and running.

### Ideas for continuing the project

The core resizer and watermarking flow is now in place, but there is plenty of room to keep building. A few directions worth exploring with more Cursor prompts:

- Apply a grid-based layout to the form so the text fields have cleaner spacing and margins.
- Add a history tab that saves processed images in the user's browser local storage, so they can revisit earlier results without re-uploading.

When prompting Cursor for follow-up changes, the same pattern applies: select the specific files the change should touch, or drag them into the chat as context, and then write a focused prompt. Keeping the context narrow and the instruction specific is usually the quickest path to a clean diff.

### Getting more out of Cursor

As projects grow, larger and more capable language models give meaningfully better results inside Cursor, both because they can reason over more code at once and because they carry more context from previous turns. Over repeated prompts, the assistant keeps track of earlier decisions and preferences, which makes each follow-up change less verbose to describe. The practical takeaway is to be deliberate about context, keep prompts specific, and let the model handle the mechanical parts while focusing your own attention on the shape of the application you want to build.



In this short conclusion, we recap what we built throughout the course and reflect on the broader lessons of vibe coding with Cursor and Flask. The goal is to summarize the journey, highlight the key takeaways, and point you toward what to explore next.

## Recap of the project

We started with the basics, building a simple Flask app to resize images. From there, we extended it step by step: we added image rotation and then styled the app to give it some personality. Each addition built on the previous one, showing how a small starting point can grow into a more capable tool with just a few focused iterations.

## Key takeaways

The bigger lesson here is that you do not need a massive plan to build something cool. A few important ideas guided the whole project:

- **Start small.** A simple Flask app was enough to get going. Once it worked, it became easier to imagine and add the next feature.
- **Get creative.** Let the tools and your own vibe guide the rest. Vibe coding works best when you stay open to new directions as you build.
- **Focus on the experience.** You are not just writing code, you are creating an experience. Styling, flow, and small touches all matter to the person using your app.

## What's next?

This course was not really about Flask or resizing images. It was about unlocking your creative coding energy and showing what is possible when you combine an AI coding assistant like Cursor with a small, focused project idea. The same approach applies to any kind of fun or useful tool you might want to build next.

If you want to learn more and create various fun and useful projects like this, [Zenva Academy](https://academy.zenva.com) is here for you. Zenva is an online learning academy with over a million students, featuring a wide range of courses for:

- People who are just starting out with coding and want a beginner-friendly path.
- People who already code and want to learn something new, from web development to game development and AI.

Thanks for joining this course. We look forward to seeing the creative projects you build next.

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

## app.py

Found in folder: /

This code defines a Flask web application that allows users to upload images, resize and rotate them, and apply a watermark. The application handles file uploads, validates the files, and processes the images according to user-specified parameters. The processed images are then saved and made available for download.

```
import os
from flask import Flask, render_template, request, send_from_directory, url_for, jsonify
from werkzeug.utils import secure_filename
from PIL import Image, ImageDraw, ImageFont

app = Flask(__name__)
app.config['UPLOAD_FOLDER'] = 'uploads'
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024 # 16MB max upload
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg'}

class WatermarkConfig:
 def __init__(self, text="ZenvaResizer", font_size=36,
 color=(255, 255, 255), opacity=128, position="center",
 rotation=0, padding=20):
 self.text = text
 self.font_size = font_size
 self.color = color
 self.opacity = opacity
 self.position = position
 self.rotation = rotation
 self.padding = padding

def apply_watermark(image, config):
 """Apply watermark to the image"""
 # Create a transparent layer for the watermark
 watermark = Image.new('RGBA', image.size, (0, 0, 0, 0))
 draw = ImageDraw.Draw(watermark)

 # Load default font
 try:
 font = ImageFont.truetype("arial.ttf", config.font_size)
 except:
 font = ImageFont.load_default()

 # Calculate text size
 text_bbox = draw.textbbox((0, 0), config.text, font=font)
 text_width = text_bbox[2] - text_bbox[0]
 text_height = text_bbox[3] - text_bbox[1]
```

```
Calculate position based on config
if config.position == "center":
 x = (image.size[0] - text_width) // 2
 y = (image.size[1] - text_height) // 2
elif config.position == "top-left":
 x = config.padding
 y = config.padding
elif config.position == "top-right":
 x = image.size[0] - text_width - config.padding
 y = config.padding
elif config.position == "bottom-left":
 x = config.padding
 y = image.size[1] - text_height - config.padding
elif config.position == "bottom-right":
 x = image.size[0] - text_width - config.padding
 y = image.size[1] - text_height - config.padding

Draw text
draw.text((x, y), config.text, font=font, fill=(*config.color, config.opacity))

Rotate watermark if needed
if config.rotation != 0:
 watermark = watermark.rotate(config.rotation, expand=True)
 # Recenter the rotated watermark
 x = (image.size[0] - watermark.size[0]) // 2
 y = (image.size[1] - watermark.size[1]) // 2
 watermark = watermark.crop((0, 0, image.size[0], image.size[1]))

Convert image to RGBA if it isn't already
if image.mode != 'RGBA':
 image = image.convert('RGBA')

Composite watermark onto image
return Image.alpha_composite(image, watermark)

Ensure upload directory exists
os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)

def allowed_file(filename):
 return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

@app.route('/')
def index():
 return render_template('index.html')

@app.route('/upload', methods=['POST'])
def upload():
 if 'images[]' not in request.files:
 return jsonify({'error': 'No files uploaded'}), 400

 files = request.files.getlist('images[]')
 if len(files) > 5:
 return jsonify({'error': 'Maximum 5 files allowed'}), 400
```



```
width = int(request.form.get('width', 800))
height = int(request.form.get('height', 600))
quality = int(request.form.get('quality', 85))
rotation = int(request.form.get('rotation', 0))

Get watermark parameters
watermark_text = request.form.get('watermark_text', 'ZenvaResizer')
font_size = int(request.form.get('font_size', 36))
color = tuple(map(int, request.form.get('color', '255,255,255').split(',')))
opacity = int(request.form.get('opacity', 128))
position = request.form.get('position', 'center')
watermark_rotation = int(request.form.get('watermark_rotation', 0))
padding = int(request.form.get('padding', 20))

Create watermark configuration
watermark_config = WatermarkConfig(
 text=watermark_text,
 font_size=font_size,
 color=color,
 opacity=opacity,
 position=position,
 rotation=watermark_rotation,
 padding=padding
)

processed_files = []
for file in files:
 if file and allowed_file(file.filename):
 filename = secure_filename(file.filename)
 input_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)
 output_filename = f'resized_{filename}'
 output_path = os.path.join(app.config['UPLOAD_FOLDER'], output_filename)

 file.save(input_path)

 with Image.open(input_path) as img:
 # Apply rotation if specified
 if rotation != 0:
 img = img.rotate(rotation, expand=True, resample=Image.Resampling
.BICUBIC)

 # Resize after rotation to maintain aspect ratio
 img = img.resize((width, height), Image.Resampling.LANCZOS)

 # Apply watermark
 img = apply_watermark(img, watermark_config)

 # Save the processed image
 img.save(output_path, quality=quality)

 processed_files.append(output_filename)
 os.remove(input_path) # Clean up original file

if request.headers.get('X-Requested-With') == 'XMLHttpRequest':
 return jsonify({'success': True, 'files': processed_files})
```

```
 return render_template('results.html', files=processed_files)

@app.route('/uploads/<filename>')
def uploaded_file(filename):
 return send_from_directory(app.config['UPLOAD_FOLDER'], filename)

if __name__ == '__main__':
 app.run(debug=True)
```

## index.html

Found in folder: **/templates**

This HTML code defines a web page for an image resizer application, including a form for users to upload images and specify resizing options, as well as a section to display the processed images. The code also includes JavaScript that handles form submission, sends a request to a server to process the images, and updates the page with the processed images. The page has a dark theme with neon-colored accents and includes a footer with links and copyright information.

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Image Resizer</title>
 <style>
 :root {
 --neon-pink: #ff2e88;
 --neon-blue: #00f3ff;
 --neon-purple: #9d00ff;
 --dark-bg: #0a0a0a;
 }

 body {
 font-family: 'Segoe UI', Arial, sans-serif;
 max-width: 800px;
 margin: 0 auto;
 padding: 20px;
 background-color: var(--dark-bg);
 color: #fff;
 line-height: 1.6;
 min-height: 100vh;
 display: flex;
 flex-direction: column;
 }

 header {
 text-align: center;
 padding: 20px 0;
 margin-bottom: 40px;
 border-bottom: 1px solid rgba(255, 255, 255, 0.1);
 }
```



```
.logo {
 font-size: 2.5em;
 font-weight: bold;
 color: var(--neon-blue);
 text-shadow: 0 0 10px var(--neon-blue),
 0 0 20px var(--neon-blue),
 0 0 30px var(--neon-blue);
 margin: 0;
 letter-spacing: 2px;
}

.tagline {
 color: var(--neon-pink);
 margin-top: 10px;
 font-size: 1.1em;
 opacity: 0.9;
}

footer {
 margin-top: auto;
 text-align: center;
 padding: 20px 0;
 border-top: 1px solid rgba(255, 255, 255, 0.1);
 color: rgba(255, 255, 255, 0.6);
}

.footer-links {
 margin: 10px 0;
}

.footer-links a {
 color: var(--neon-blue);
 text-decoration: none;
 margin: 0 10px;
 transition: all 0.3s ease;
}

.footer-links a:hover {
 color: var(--neon-pink);
 text-shadow: 0 0 10px var(--neon-pink);
}

.form-group {
 margin-bottom: 25px;
 background: rgba(255, 255, 255, 0.05);
 padding: 20px;
 border-radius: 8px;
 border: 1px solid rgba(255, 255, 255, 0.1);
 transition: all 0.3s ease;
}

.form-group:hover {
 border-color: var(--neon-purple);
 box-shadow: 0 0 15px rgba(157, 0, 255, 0.3);
}
```

```
label {
 display: block;
 margin-bottom: 10px;
 color: var(--neon-pink);
 font-weight: 500;
}

input[type="number"],
input[type="text"],
select {
 width: 100%;
 padding: 8px 12px;
 background: rgba(255, 255, 255, 0.1);
 border: 1px solid var(--neon-blue);
 border-radius: 4px;
 color: #fff;
 font-size: 16px;
 margin-bottom: 10px;
}

input[type="number"]:focus,
input[type="text"]:focus,
select:focus {
 outline: none;
 box-shadow: 0 0 10px var(--neon-blue);
}

input[type="file"] {
 background: rgba(255, 255, 255, 0.1);
 padding: 10px;
 border-radius: 4px;
 border: 1px dashed var(--neon-blue);
 color: #fff;
 width: 100%;
}

input[type="file"]::-webkit-file-upload-button {
 background: var(--neon-purple);
 color: white;
 padding: 8px 16px;
 border: none;
 border-radius: 4px;
 cursor: pointer;
 margin-right: 10px;
}

input[type="color"] {
 width: 100%;
 height: 40px;
 padding: 2px;
 background: rgba(255, 255, 255, 0.1);
 border: 1px solid var(--neon-blue);
 border-radius: 4px;
 cursor: pointer;
}
```

```
}

input[type="range"] {
 width: 100%;
 height: 8px;
 background: rgba(255, 255, 255, 0.1);
 border-radius: 4px;
 outline: none;
 -webkit-appearance: none;
}

input[type="range"]::-webkit-slider-thumb {
 -webkit-appearance: none;
 width: 20px;
 height: 20px;
 background: var(--neon-purple);
 border-radius: 50%;
 cursor: pointer;
 transition: all 0.3s ease;
}

input[type="range"]::-webkit-slider-thumb:hover {
 background: var(--neon-pink);
 box-shadow: 0 0 10px var(--neon-pink);
}

button {
 background: linear-gradient(45deg, var(--neon-purple), var(--neon-pink));
 color: white;
 padding: 12px 30px;
 border: none;
 border-radius: 25px;
 cursor: pointer;
 font-size: 16px;
 font-weight: bold;
 width: 100%;
 text-transform: uppercase;
 letter-spacing: 1px;
 transition: all 0.3s ease;
 box-shadow: 0 0 15px rgba(157, 0, 255, 0.3);
}

button:hover {
 transform: translateY(-2px);
 box-shadow: 0 0 20px rgba(157, 0, 255, 0.5);
}

.container {
 max-width: 800px;
 margin: 0 auto;
 padding: 20px;
 flex: 1;
}

.watermark-controls {
```



```
 background: rgba(255, 255, 255, 0.05);
 padding: 20px;
 border-radius: 8px;
 border: 1px solid rgba(255, 255, 255, 0.1);
 margin-bottom: 20px;
 transition: all 0.3s ease;
 }

 .watermark-controls:hover {
 border-color: var(--neon-purple);
 box-shadow: 0 0 15px rgba(157, 0, 255, 0.3);
 }

 .watermark-controls h4 {
 color: var(--neon-blue);
 margin-bottom: 20px;
 text-shadow: 0 0 10px var(--neon-blue);
 }

 .preview-container {
 max-width: 300px;
 margin: 20px auto;
 position: relative;
 }

 .preview-image {
 max-width: 100%;
 border: 1px solid var(--neon-blue);
 border-radius: 4px;
 box-shadow: 0 0 10px rgba(0, 243, 255, 0.3);
 }

 .results {
 margin-top: 30px;
 }

 .results h3 {
 color: var(--neon-blue);
 text-shadow: 0 0 10px var(--neon-blue);
 margin-bottom: 20px;
 }

 .download-btn {
 background: linear-gradient(45deg, var(--neon-purple), var(--neon-pink));
 color: white;
 padding: 8px 20px;
 border: none;
 border-radius: 20px;
 cursor: pointer;
 font-size: 14px;
 text-decoration: none;
 display: inline-block;
 margin-top: 10px;
 transition: all 0.3s ease;
 }
```

```
.download-btn:hover {
 transform: translateY(-2px);
 box-shadow: 0 0 15px rgba(157, 0, 255, 0.5);
}

.opacity-value {
 color: var(--neon-pink);
 text-align: center;
 margin-top: 5px;
 font-size: 14px;
}
</style>
</head>
<body>
 <header>
 <h1 class="logo">ZenvaResizer</h1>
 <p class="tagline">Transform your images with neon precision</p>
 </header>

 <div class="container">
 <form id="uploadForm" enctype="multipart/form-data">
 <div class="form-group">
 <label for="images">Select Images (Max 5)</label>
 <input type="file" id="images" name="images[]" multiple accept="image
/*" required>
 </div>

 <div class="form-group">
 <label for="width">Width (px)</label>
 <input type="number" id="width" name="width" value="800" required>
 </div>

 <div class="form-group">
 <label for="height">Height (px)</label>
 <input type="number" id="height" name="height" value="600" required>
 </div>

 <div class="form-group">
 <label for="quality">Quality (1-100)</label>
 <input type="number" id="quality" name="quality" value="85" min="1" m
ax="100" required>
 </div>

 <div class="form-group">
 <label for="rotation">Rotation (degrees)</label>
 <input type="number" id="rotation" name="rotation" value="0" min="0"
max="360">
 </div>

 <div class="watermark-controls">
 <h4>Watermark Settings</h4>

 <div class="form-group">
 <label for="watermark_text">Watermark Text</label>
```

```
 <input type="text" id="watermark_text" name="watermark_text" placeholder="Enter watermark text">
 </div>

 <div class="form-group">
 <label for="font_size">Font Size</label>
 <input type="number" id="font_size" name="font_size" value="36" min="8" max="200">
 </div>

 <div class="form-group">
 <label for="color">Color</label>
 <input type="color" id="color" name="color" value="#ffffff">
 </div>

 <div class="form-group">
 <label for="opacity">Opacity (0-255)</label>
 <input type="range" id="opacity" name="opacity" min="0" max="255" value="128">
 <div class="opacity-value" id="opacityValue">128</div>
 </div>

 <div class="form-group">
 <label for="position">Position</label>
 <select id="position" name="position">
 <option value="center">Center</option>
 <option value="top-left">Top Left</option>
 <option value="top-right">Top Right</option>
 <option value="bottom-left">Bottom Left</option>
 <option value="bottom-right">Bottom Right</option>
 </select>
 </div>

 <div class="form-group">
 <label for="watermark_rotation">Watermark Rotation (degrees)</label>
 <input type="number" id="watermark_rotation" name="watermark_rotation" value="0" min="0" max="360">
 </div>

 <div class="form-group">
 <label for="padding">Padding (px)</label>
 <input type="number" id="padding" name="padding" value="20" min="0" max="100">
 </div>

 <button type="submit">Process Images</button>
</form>

<div id="results" class="results"></div>
</div>

<footer>
 <div class="footer-links">
```



```
 Home
 GitHub
 Pillow Docs
 </div>
 <p>© 2025 ZenvaResizer. All rights reserved.</p>
</footer>

<script>
 document.getElementById('opacity').addEventListener('input', function() {
 document.getElementById('opacityValue').textContent = this.value;
 });

 document.getElementById('uploadForm').addEventListener('submit', function(e)
{
 e.preventDefault();

 const formData = new FormData(this);
 const color = document.getElementById('color').value;
 formData.set('color', `${parseInt(color.substr(1,2), 16)},${parseInt(color
r.substr(3,2), 16)},${parseInt(color.substr(5,2), 16)}`);

 fetch('/upload', {
 method: 'POST',
 body: formData,
 headers: {
 'X-Requested-With': 'XMLHttpRequest'
 }
 })
 .then(response => response.json())
 .then(data => {
 if (data.success) {
 const resultsDiv = document.getElementById('results');
 resultsDiv.innerHTML = '<h3>Processed Images:</h3>';
 data.files.forEach(file => {
 resultsDiv.innerHTML += `
 <div class="form-group">

 <a href="/uploads/${file}" download class="download-
btn">Download
 </div>
 `;
 });
 } else {
 alert('Error: ' + data.error);
 }
 })
 .catch(error => {
 console.error('Error:', error);
 alert('An error occurred while processing the images.');
```

## results.html

Found in folder: **/templates**

This HTML code defines the structure and styling of a web page for displaying resized images. It includes a header with a logo and tagline, a container for displaying the resized images with download links, and a footer with navigation links and copyright information. The page uses CSS styles to create a neon-themed design with custom colors and effects.

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Resized Images - ZenvaResizer</title>
 <style>
 :root {
 --neon-pink: #ff2e88;
 --neon-blue: #00f3ff;
 --neon-purple: #9d00ff;
 --dark-bg: #0a0a0a;
 }

 body {
 font-family: 'Segoe UI', Arial, sans-serif;
 max-width: 800px;
 margin: 0 auto;
 padding: 20px;
 background-color: var(--dark-bg);
 color: #fff;
 line-height: 1.6;
 min-height: 100vh;
 display: flex;
 flex-direction: column;
 }

 header {
 text-align: center;
 padding: 20px 0;
 margin-bottom: 40px;
 border-bottom: 1px solid rgba(255, 255, 255, 0.1);
 }

 .logo {
 font-size: 2.5em;
 font-weight: bold;
 color: var(--neon-blue);
 text-shadow: 0 0 10px var(--neon-blue),
 0 0 20px var(--neon-blue),
 0 0 30px var(--neon-blue);
 margin: 0;
 letter-spacing: 2px;
 }
 </style>
</head>
<body>
 <div class="container">
 <div class="header">
 <div class="logo">ZENVA</div>
 <div class="tagline">Vibe Coding with Cursor</div>
 </div>
 <div class="main">
 <div class="image-container">

 <div class="download-links">
 Download
 Share
 </div>
 </div>
 </div>
 <div class="footer">
 <div class="nav-links">
 Home
 About
 Contact
 </div>
 <div class="copyright">
 © 2025 Zenva Pty Ltd. All rights reserved.
 </div>
 </div>
 </div>
</body>
</html>
```



```
.tagline {
 color: var(--neon-pink);
 margin-top: 10px;
 font-size: 1.1em;
 opacity: 0.9;
}

footer {
 margin-top: auto;
 text-align: center;
 padding: 20px 0;
 border-top: 1px solid rgba(255, 255, 255, 0.1);
 color: rgba(255, 255, 255, 0.6);
}

.footer-links {
 margin: 10px 0;
}

.footer-links a {
 color: var(--neon-blue);
 text-decoration: none;
 margin: 0 10px;
 transition: all 0.3s ease;
}

.footer-links a:hover {
 color: var(--neon-pink);
 text-shadow: 0 0 10px var(--neon-pink);
}

.image-container {
 margin: 30px 0;
 padding: 20px;
 background: rgba(255, 255, 255, 0.05);
 border: 1px solid rgba(255, 255, 255, 0.1);
 border-radius: 8px;
 transition: all 0.3s ease;
}

.image-container:hover {
 border-color: var(--neon-purple);
 box-shadow: 0 0 20px rgba(157, 0, 255, 0.3);
 transform: translateY(-2px);
}

.image-container img {
 max-width: 100%;
 height: auto;
 border-radius: 4px;
 box-shadow: 0 0 15px rgba(0, 243, 255, 0.2);
}

.download-link {
```

```
 display: inline-block;
 background: linear-gradient(45deg, var(--neon-purple), var(--neon-pink));
 color: white;
 padding: 12px 30px;
 text-decoration: none;
 border-radius: 25px;
 margin-top: 15px;
 font-weight: bold;
 text-transform: uppercase;
 letter-spacing: 1px;
 transition: all 0.3s ease;
 box-shadow: 0 0 15px rgba(157, 0, 255, 0.3);
 }

 .download-link:hover {
 transform: translateY(-2px);
 box-shadow: 0 0 20px rgba(157, 0, 255, 0.5);
 }

 .back-link {
 display: inline-block;
 margin-bottom: 30px;
 color: var(--neon-blue);
 text-decoration: none;
 font-weight: 500;
 transition: all 0.3s ease;
 padding: 8px 16px;
 border-radius: 4px;
 background: rgba(0, 243, 255, 0.1);
 }

 .back-link:hover {
 color: var(--neon-pink);
 background: rgba(255, 46, 136, 0.1);
 text-shadow: 0 0 10px var(--neon-pink);
 }

 .container {
 max-width: 600px;
 margin: 0 auto;
 padding: 20px;
 flex: 1;
 }

 .page-title {
 text-align: center;
 color: var(--neon-blue);
 text-shadow: 0 0 10px var(--neon-blue),
 0 0 20px var(--neon-blue),
 0 0 30px var(--neon-blue);
 margin-bottom: 40px;
 }
}
</style>
</head>
<body>
```

```
<header>
 <h1 class="logo">ZenvaResizer</h1>
 <p class="tagline">Transform your images with neon precision</p>
</header>

<div class="container">
 ? Back to Upload
 <h2 class="page-title">Resized Images</h2>

 {% for file in files %}
 <div class="image-container">

 Download
 </div>
 {% endfor %}
</div>

<footer>
 <div class="footer-links">
 Home
 GitHub
 Pillow Docs
 </div>
 <p>© 2024 ZenvaResizer. All rights reserved.</p>
</footer>
</body>
</html>
```

## requirements.txt

Found in folder: /

This text document lists the dependencies required for a project, specifying the exact version of each library. The dependencies include Flask, Pillow, Werkzeug, and requests. The versions of these libraries are pinned to ensure consistency and reproducibility.

```
Flask==2.0.1
Pillow==9.0.0
Werkzeug==2.0.1
requests==2.31.0
```

## README.md

Found in folder: /

This is a Cursor generated README file written in Markdown. It provides information about the ZenvaResizer project, including its features, requirements, installation, usage, API endpoints, and license. The file serves as a guide for users and developers who want to use or contribute to the

project.

# ZenvaResizer

A modern, neon-themed web application for batch image processing with a sleek user interface. Built with Flask and Pillow, ZenvaResizer allows you to resize, rotate, and optimize multiple images simultaneously.

## ## Features

- ?? **Batch Processing**: Upload and process up to 5 images at once
- ? **Custom Dimensions**: Set custom width and height for your images
- ? **Rotation**: Rotate images by any angle (0-359 degrees)
- ? **Quality Control**: Adjust output image quality (1-100)
- ? **Modern UI**: Sleek neon-themed interface with smooth animations
- ? **Fast Processing**: Efficient image processing using Pillow
- ? **Responsive Design**: Works on both desktop and mobile devices

## ## Requirements

- Python 3.7+
- Flask
- Pillow (PIL)
- Werkzeug

## ## Installation

1. Clone the repository:

```
```bash
git clone https://github.com/yourusername/zenvaresizer.git
cd zenvaresizer
```
```

2. Create and activate a virtual environment:

```
```bash
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
```
```

3. Install dependencies:

```
```bash
pip install -r requirements.txt
```
```

## ## Usage

1. Start the application:

```
```bash
python app.py
```
```

2. Open your web browser and navigate to `http://localhost:5000`

3. Use the web interface to:

- Select up to 5 images

- Set desired dimensions
- Adjust quality settings
- Apply rotation if needed
- Process and download the results

### ## API Endpoints

- `GET /``: Main application interface
- `POST /upload``: Process uploaded images
  - Parameters:
    - `images[]``: Image files (max 5)
    - `width``: Target width in pixels
    - `height``: Target height in pixels
    - `quality``: Output quality (1-100)
    - `rotation``: Rotation angle in degrees (0-359)
- `GET /uploads/<filename>``: Access processed images

### ## Supported Image Formats

- PNG
- JPEG/JPG

### ## Contributing

1. Fork the repository
2. Create your feature branch (`git checkout -b feature/AmazingFeature``)
3. Commit your changes (`git commit -m 'Add some AmazingFeature'``)
4. Push to the branch (`git push origin feature/AmazingFeature``)
5. Open a Pull Request

### ## License

This project is licensed under the MIT License - see the [LICENSE](LICENSE) file for details.

### ## Acknowledgments

- [Flask](https://flask.palletsprojects.com/)
- [Pillow](https://python-pillow.org/)
- [Werkzeug](https://werkzeug.palletsprojects.com/)

### ## Contact

Your Name - [@yourtwitter](https://twitter.com/yourtwitter)

Project Link: [https://github.com/yourusername/zenvaresizer](https://github.com/yourusername/zenvaresizer)